King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS 211: Digital Logic Design

Chapter 5 : Designing Compinational Systems

Part I (5.1 – 5.3)

Outline

- Combinational systems
- Iterative systems
- Adders, subtractors and comparators
- Decoders
- Encoders and Priority Encoders

Combinational Systems

- In combinational systems, the output at any instant of time depends only on what the inputs are at that time

- Large systems
  - Usually designed by breaking them up into smaller subsystems
  - In the process of designing large systems, we shall first look at the systems that consist of a number of identical blocks
  - Sometimes referred to as *iterative systems.*
  - Examples: Adders and other arithmetic functions
  - Next, we will look at some common types of circuit
    - Binary decoder and encoder, and multiplexer

Iterative systems

- We will first look at adders as an example of a system that can be implemented with multiple copies of a smaller circuit

- Because signals in large systems pass through many layers of logic, the small delay encountered as a signal passes through a single gate adds up

- We will use the design of a multibit adder to illustrate this

Full Adder

• When add two numbers by hand, add the two least significant digits

• (plus possibly carry-in) to produce one bit of **sum** and a **carry** to next bit

• A full adder (*FA*) is a combinational circuit that forms the arithmetic sum of three input bits – Consists of

- 3 inputs
- 2 outputs

Full Adder cont.

- Two of the input variables, denoted by *A* and *B* represent the two significant bits to be added

- Third input *C* represents carry from previous lower significant position

- Two outputs are designated by the symbols, *S* (sum) and C_{out} (carry out)

Full Adder cont.

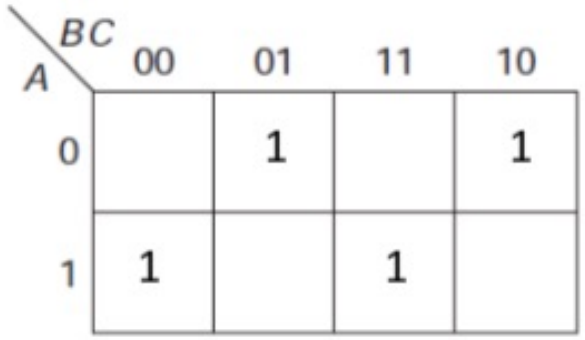
If we wish to design a full adder using NAND gates, we shall first construct TT as follows:

Table 1.5 One-bit adder.

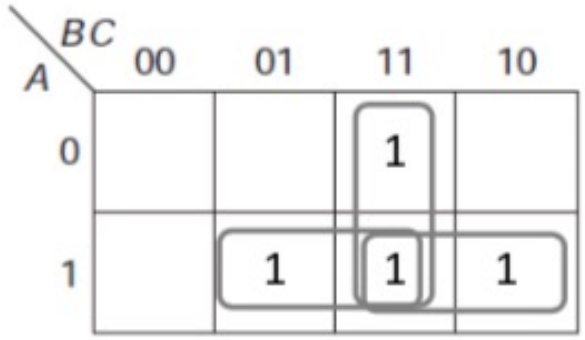
a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder cont.

- K-maps for S and C_{out} are as follows:



$S = A'B'C + A'BC' + AB'C' + ABC$



$C_{out} = AB + BC + AC$

Table 1.5 One-bit adder.

a	b	c_{in}	c_{out}	s
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

- AND-OR implementation of S and C_{out} are left as exercise for students

Full Adder cont.

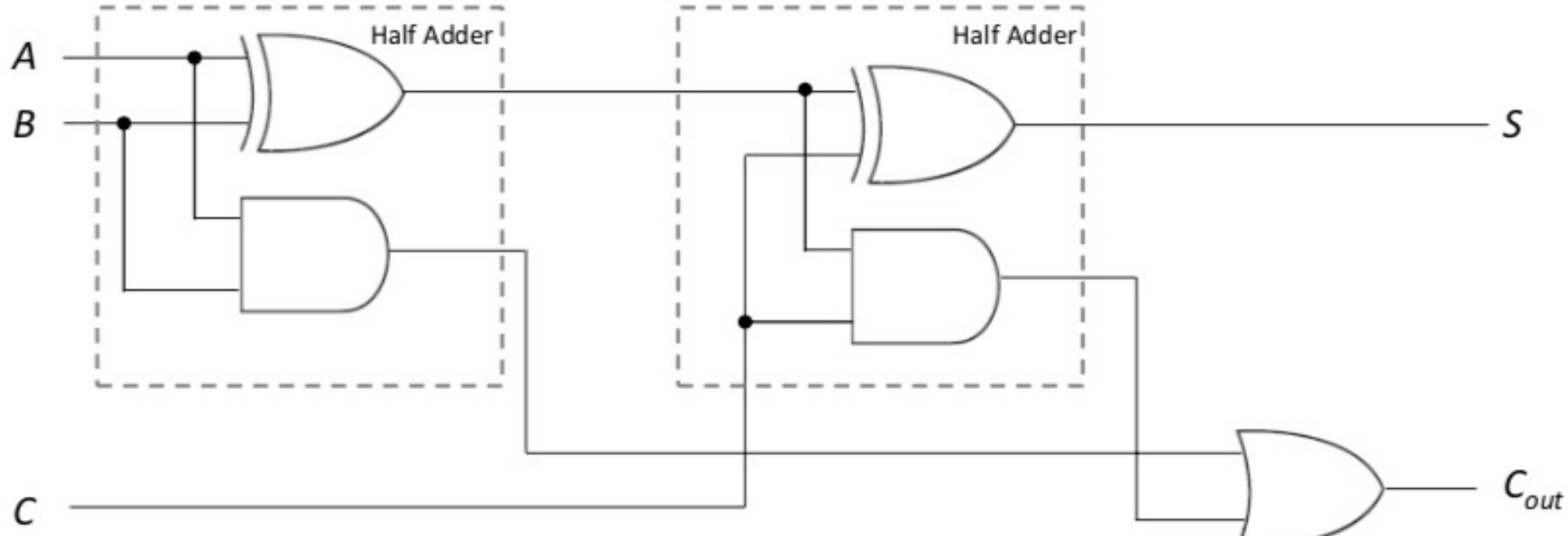
Now, we can simplify the functions for S and C_{out} as

$$S = A'B'C + ABC + A'BC' + AB'C'$$
$$= C(A'B' + AB) + C'(A'B + AB')$$
$$= C(A \oplus B)' + C'(A \oplus B)$$
$$= C \oplus (A \oplus B)$$

$$C_{out} = AB + AC + BC$$
$$= AB + AC.(B+B') + BC.(A + A')$$
$$= AB + ABC + AB'C + ABC + A'BC$$
$$= AB + ABC + AB'C + A'BC$$
$$= AB(1 + C) + C(AB' + A'B)$$
$$= AB + C(A \oplus B)$$

Full Adder cont.

Implementation of full adder with two half-adders and one OR gate is as follows:

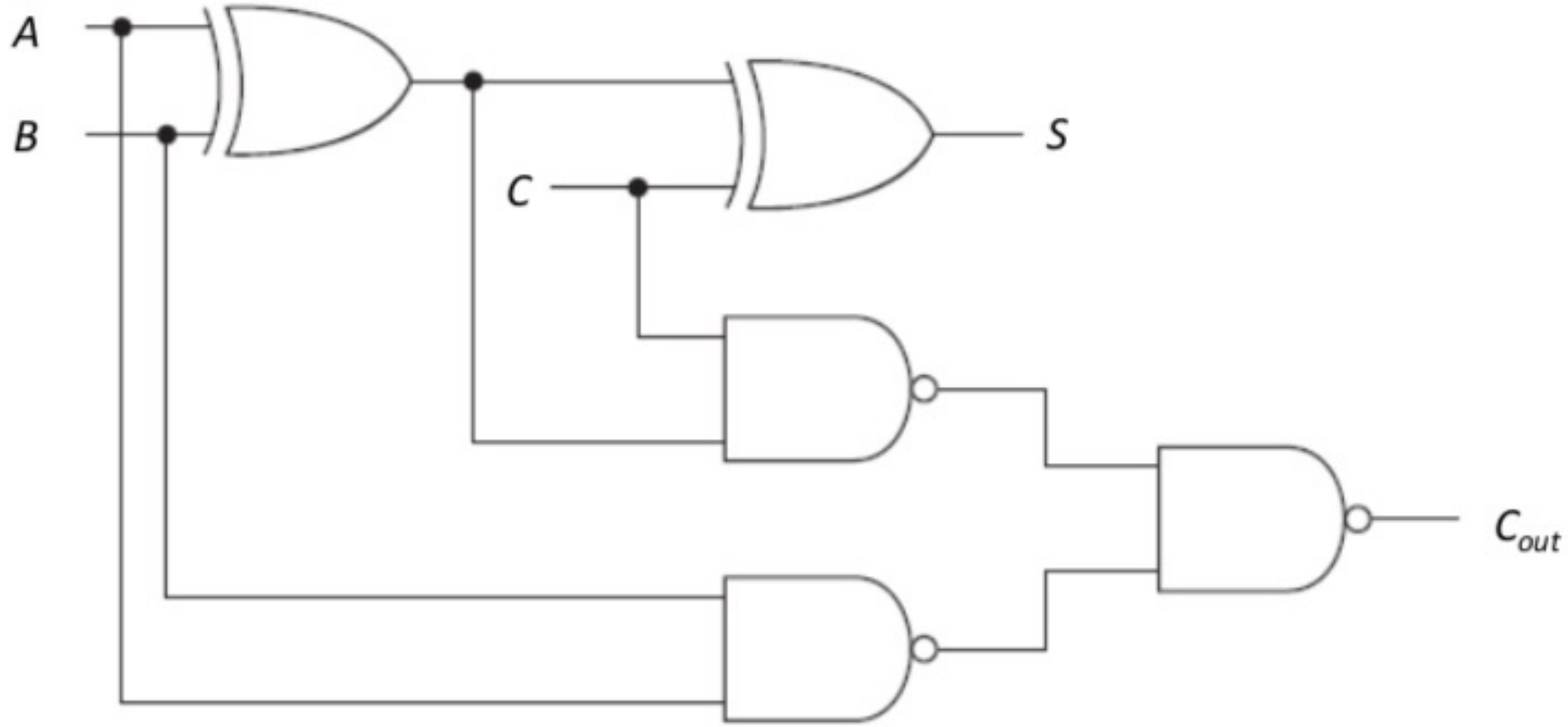


$$S = C \oplus (A \oplus B)$$

$$C_{out} = AB + C(A \oplus B)$$

Full Adder cont.

Implementation of full adder with Exclusive-OR and NAND gates is as follows:

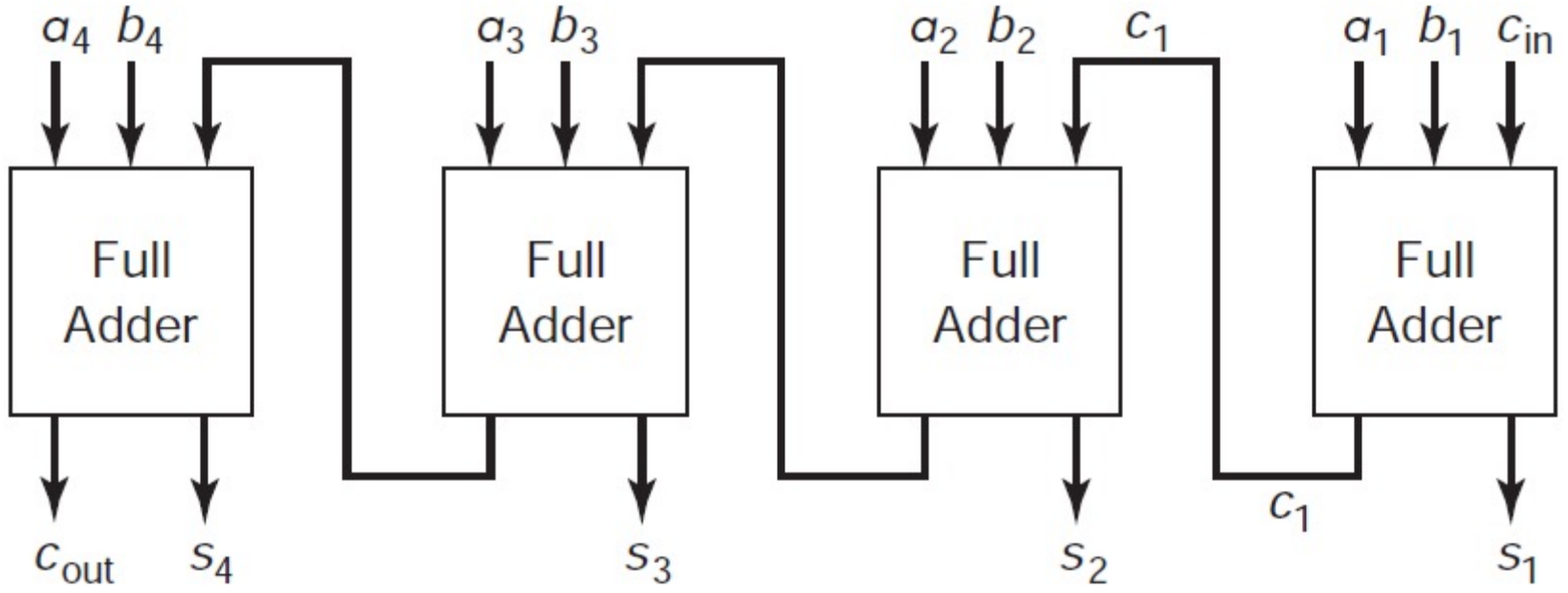


4-bit Adder

- If we wish to build an *n*-bit adder, we need only connect *n* of these.

- A 4-bit version is shown

Figure 5.1 A 4-bit adder.



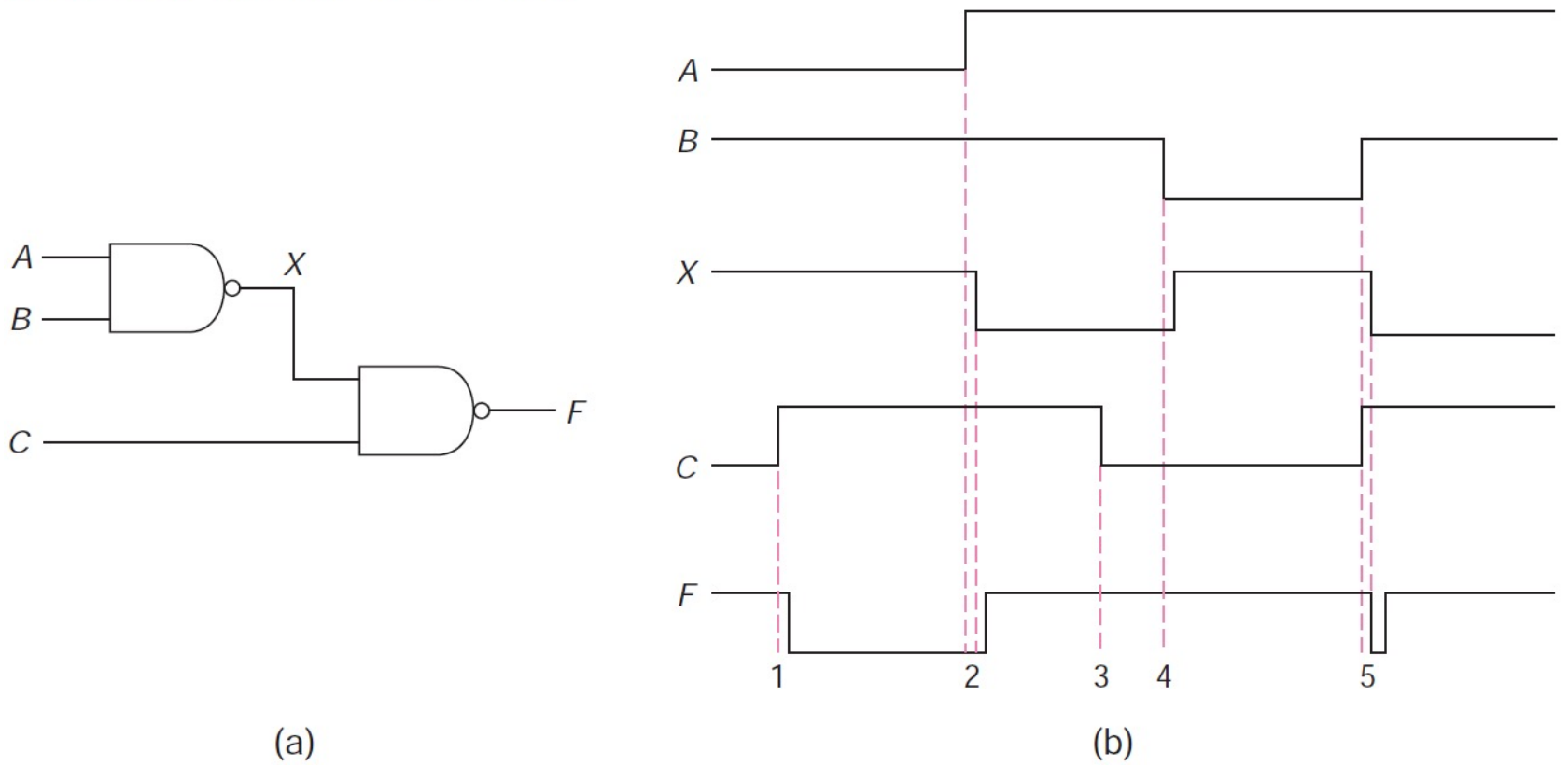
Delay on Combinational Logic Circuits

- When the input to a gate changes, the output of that gate does not change instantaneously
- There is a small delay, denoted by Δ
- If output of one gate is used as input to another, delays add

Delay on Combinational Logic Circuits

Block diagram of circuit and timing diagram associated with it

Figure 5.2 Illustration of gate delay.



Delay Through 1-bit Adder

- We will look at the time it takes for the result of an addition to be available at the two outputs of *FA*

- We will assume that all inputs are available at the same time

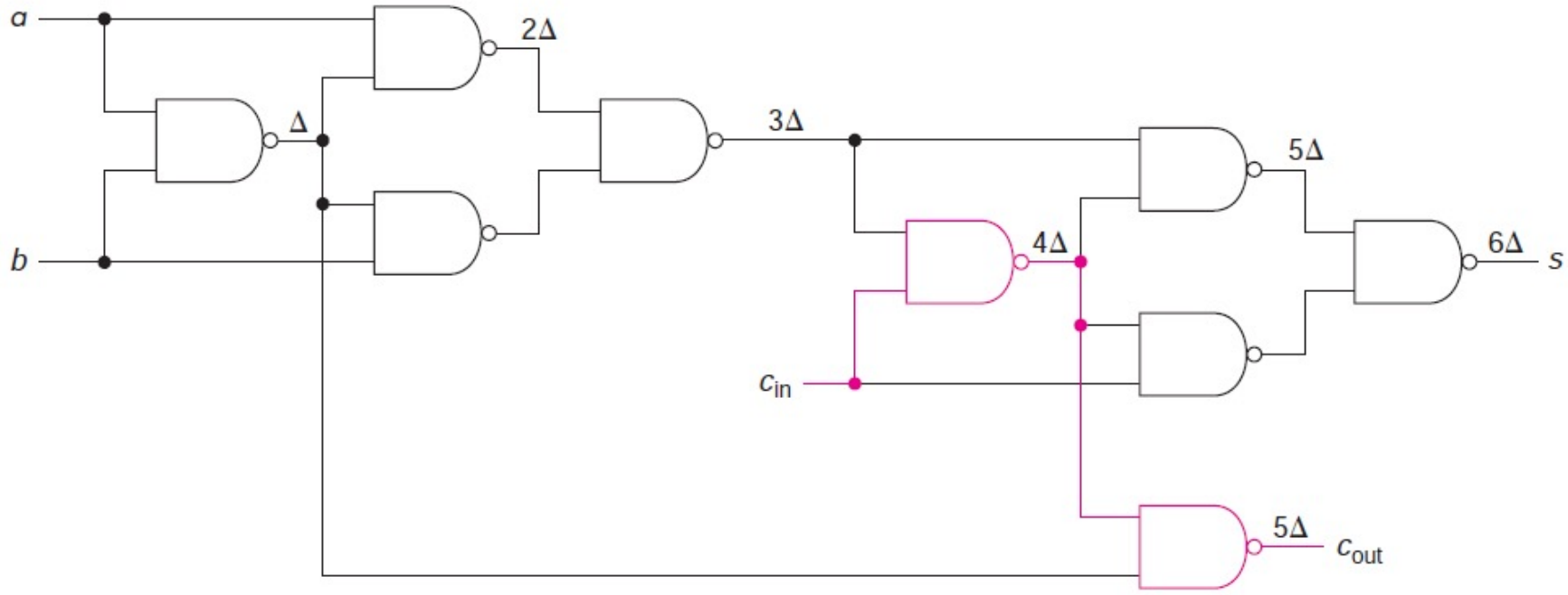
- Next Figure shows the delay (from when inputs a and b change) indicated at various points in the circuit

- Of course, if two inputs to a gate change at different times, the output may change as late as Δ after the last input change

Delay Through 1-bit Adder

Delay from the time inputs *a or b change to the time* that the sum is available is 6Δ and to the time that the carry-out is available is 5Δ

Figure 5.3 Delay through a 1-bit adder.

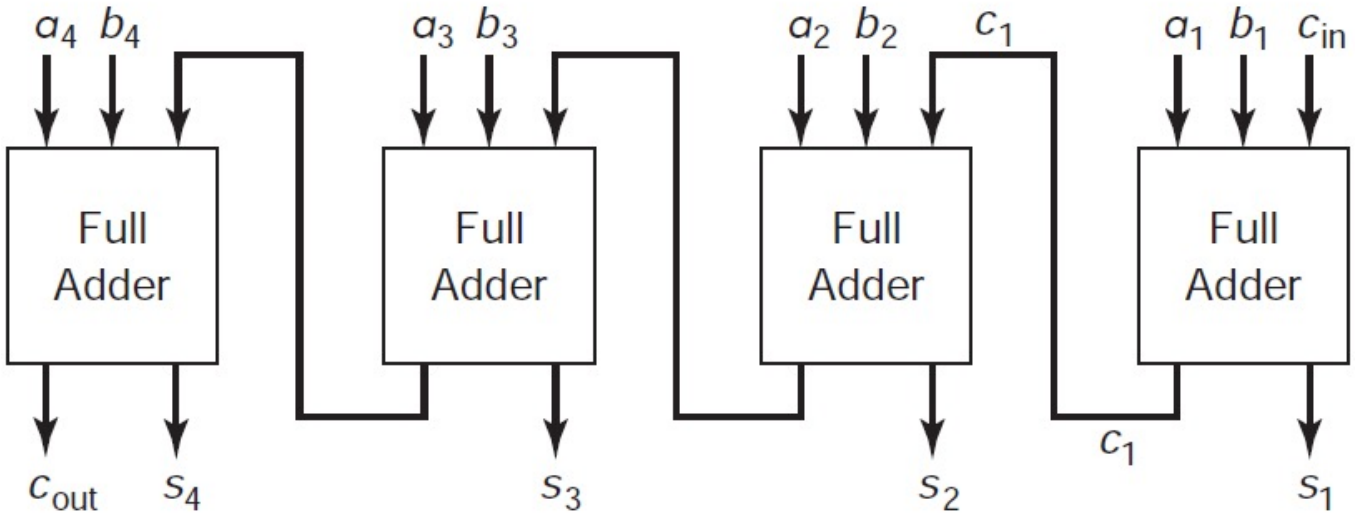


Conclusion

If a and b are established

- All of bits of two multidigit numbers to be added are available at one time
- Thus, after the least significant digit, all of the *a's and b's are established before Cin arrives*

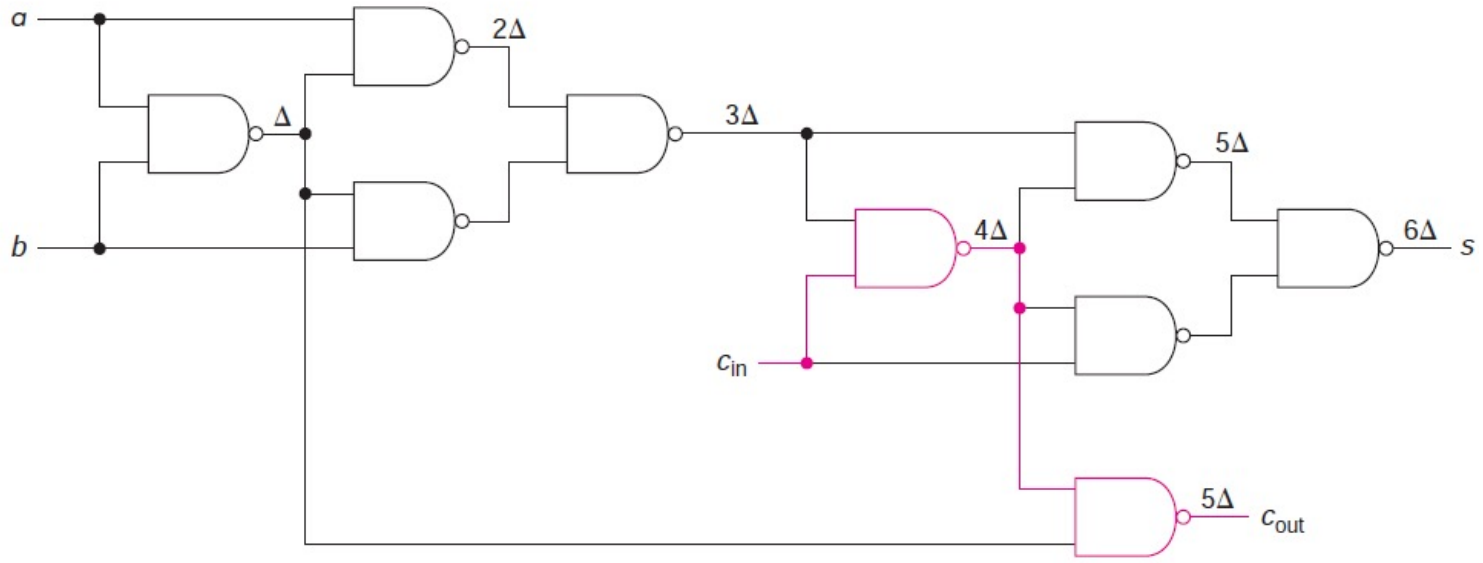
Figure 5.1 A 4-bit adder.



Conclusion

- Delay from *cin to cout* is only 2Δ
  - Since *cin* passes through only two gates on the way to *c*out , as shown in the colored path
- Delay from *cin* to *s* is 3Δ

Figure 5.3 Delay through a 1-bit adder.



Conclusion

- We can build an *n*-bit adder with *n* copies of the *FA*

- Total time required is calculated as
  - delay from *inputs* to c_{out} (for LSB): 5Δ
  - plus $n-2$ times delay from c_{in} to c_{out} (for middle full adders): $(n-2)2\Delta$
  - plus the longer of delay from *cin* to *cout* or from *cin* to *s* (for MSB): 3Δ

- For the multilevel adder, that equals

$$5\Delta + 2(n-2)\Delta + 3\Delta = 5\Delta + 2n\Delta - 4\Delta + 3\Delta = 4\Delta + 2n\Delta = (2n+4)\Delta$$

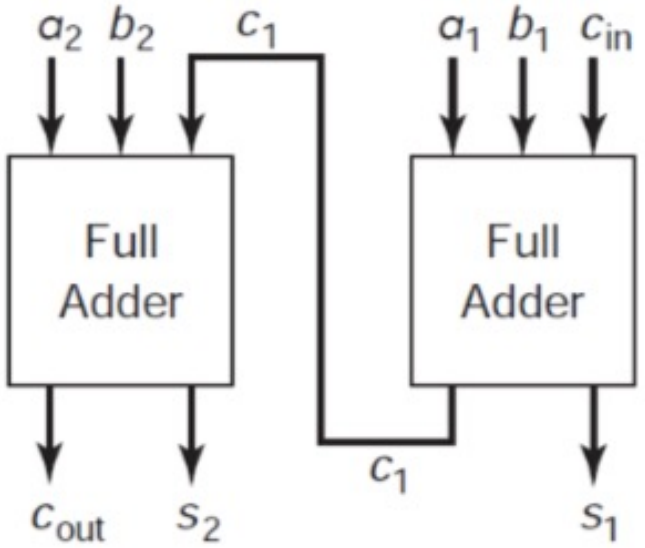
- For a 64-bit adder, the delay would be: $(2 \times 64 + 4)\Delta = 132\Delta$

Adder

- One approach to building an *n*-bit adder is to connect together *n* 1-bit adders
  - Referred to as **a carry ripple** adder
- Time for output of adder to become stable may be as large as $(2n + 4)\Delta$
- To speed this up, several approaches have been attempted
- One approach is to implement a multi-bit adder with an SOP expression

2-Bit Adder

- TT for 2-bit adder is shown, where bit 1 is the lower order bit
- Can be implemented by an SOP expression using K-maps



$$\begin{array}{rrr} & & c_{in} \\ & a_2 & a_1 \\ + & b_2 & b_1 \\ \hline c_{out} & s_2 & s_1 \end{array}$$

Table 5.1 Two-bit adder truth table.

a_2	b_2	a_1	b_1	c_{in}	c_{out}	s_2	s_1
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	0	1	0
0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0
0	0	1	1	0	0	1	0
0	0	1	1	1	0	1	1
0	1	0	0	0	0	1	0
0	1	0	0	1	0	1	1
0	1	0	1	0	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	0	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	0	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	0	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	0	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1

2-Bit Adder cont.

- Examples:

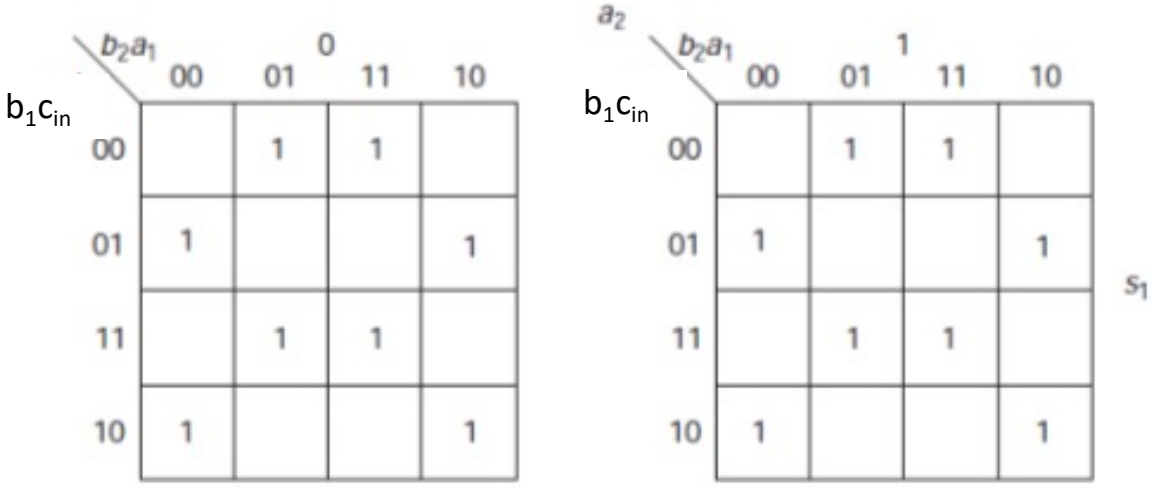
			①			②			③		
		c_{in}			0		1	1		1	1
	a_2	a_1		0	1		1	0		1	1
+	b_2	b_1	+	0	0	+	0	1	+	1	1
c_{out}	s_2	s_1	0	0	1	1	0	0	1	1	1

Table 5.1 Two-bit adder truth table.

	a_2	b_2	a_1	b_1	c_{in}	c_{out}	s_2	s_1
	0	0	0	0	0	0	0	0
	0	0	0	0	1	0	0	1
	0	0	0	1	0	0	0	1
	0	0	0	1	1	0	1	0
①	0	0	1	0	0	0	0	1
	0	0	1	0	1	0	1	0
	0	0	1	1	0	0	1	0
	0	0	1	1	1	0	1	1
	0	1	0	0	0	0	1	0
	0	1	0	0	1	0	1	1
	0	1	0	1	0	0	1	1
	0	1	0	1	1	1	0	0
②	0	1	1	0	0	0	1	1
	0	1	1	0	1	1	0	0
	0	1	1	1	0	1	0	0
	0	1	1	1	1	1	0	1
	1	0	0	0	0	0	1	0
	1	0	0	0	1	0	1	1
	1	0	0	1	0	0	1	1
	1	0	0	1	1	1	0	0
	1	0	1	0	0	0	1	1
	1	0	1	0	1	1	0	0
	1	0	1	1	0	1	0	0
	1	0	1	1	1	1	0	1
	1	1	0	0	0	1	0	0
	1	1	0	0	1	1	0	1
	1	1	0	1	0	1	0	1
	1	1	0	1	1	1	1	0
	1	1	1	0	0	1	0	1
	1	1	1	0	1	1	1	0
	1	1	1	1	0	1	1	0
③	1	1	1	1	1	1	1	1

2-Bit Adder cont.

- K-map for s_1 is as follows:



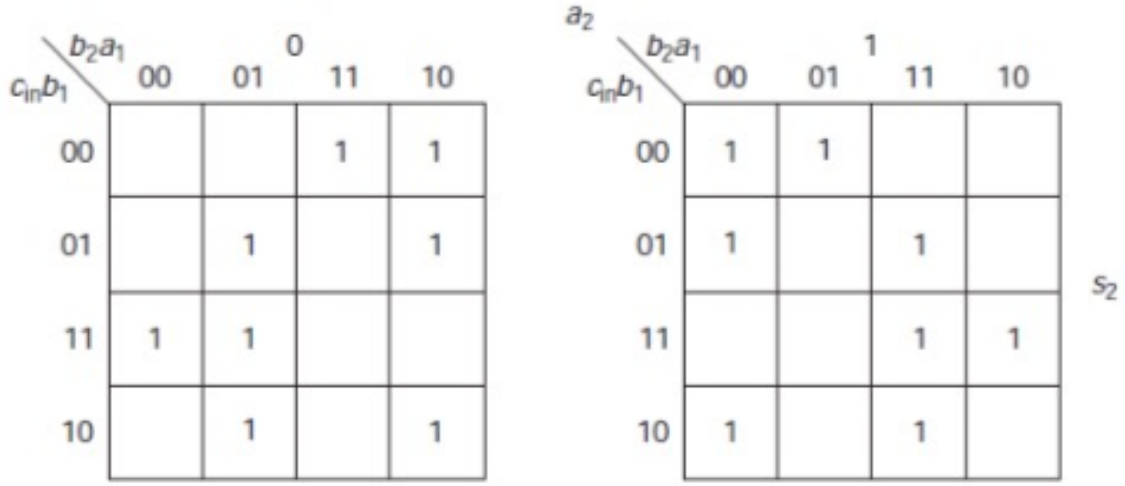
$s_1 = a_1'b_1c_{in}' + a_1b_1'c_{in}' + a_1'b_1'c_{in} + a_1b_1c_{in}$

Table 5.1 Two-bit adder truth table.

a_2	b_2	a_1	b_1	c_{in}	c_{out}	s_2	s_1
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	0	1	0
0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0
0	0	1	1	0	0	1	0
0	0	1	1	1	0	1	1
0	1	0	0	0	0	1	0
0	1	0	0	1	0	1	1
0	1	0	1	0	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	0	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	0	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	0	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	0	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1

2-Bit Adder cont.

- K-map for s_2 is as follows:



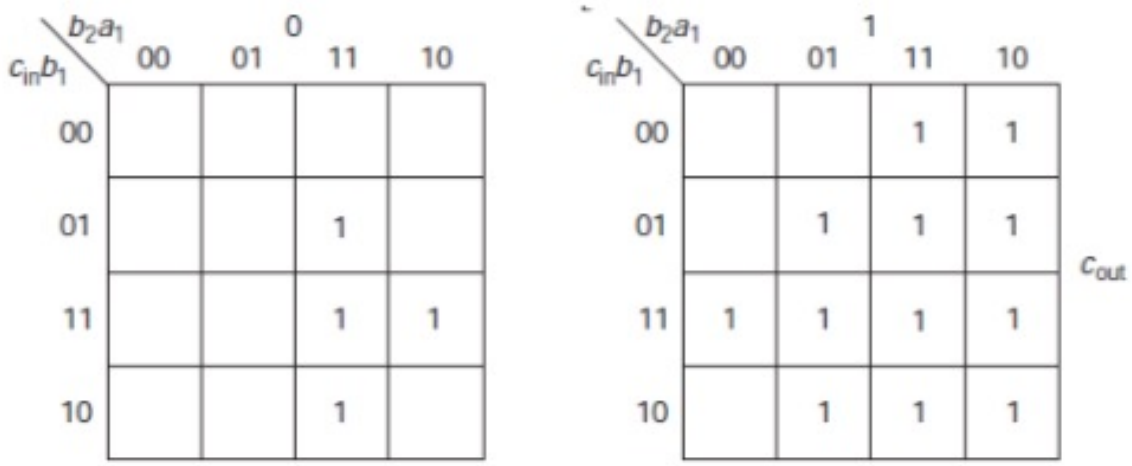
$$s_2 = a_1b_1a_2'b_2' + a_1b_1a_2b_2 + c_{in}'a_1'a_2'b_2 + c_{in}'a_1'a_2b_2' + c_{in}'b_1'a_2'b_2 + c_{in}'b_1'a_2b_2' + a_1'b_1'a_2b_2' + a_1'b_1'a_2'b_2 + c_{in}b_1a_2'b_2' + c_{in}b_1a_2b_2 + c_{in}a_1a_2'b_2' + c_{in}a_1a_2b_2$$

Table 5.1 Two-bit adder truth table.

a_2	b_2	a_1	b_1	c_{in}	c_{out}	s_2	s_1
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	0	1	0
0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0
0	0	1	1	0	0	1	0
0	0	1	1	1	0	1	1
0	1	0	0	0	0	1	0
0	1	0	0	1	0	1	1
0	1	0	1	0	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	0	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	0	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	0	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	0	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1

2-Bit Adder cont.

K-map for c_{out} is as follows:



$$c_{out} = a_2b_2 + a_1b_1a_2 + a_1b_1b_2 + c_{in}b_1b_2 + c_{in}b_1a_2 + c_{in}a_1b_2 + c_{in}a_1a_2$$

Table 5.1 Two-bit adder truth table.

a_2	b_2	a_1	b_1	c_{in}	c_{out}	s_2	s_1
0	0	0	0	0	0	0	0
0	0	0	0	1	0	0	1
0	0	0	1	0	0	0	1
0	0	0	1	1	0	1	0
0	0	1	0	0	0	0	1
0	0	1	0	1	0	1	0
0	0	1	1	0	0	1	0
0	0	1	1	1	0	1	1
0	1	0	0	0	0	1	0
0	1	0	0	1	0	1	1
0	1	0	1	0	0	1	1
0	1	0	1	1	1	0	0
0	1	1	0	0	0	1	1
0	1	1	0	1	1	0	0
0	1	1	1	0	1	0	0
0	1	1	1	1	1	0	1
1	0	0	0	0	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	0	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	0	0	1	1
1	0	1	0	1	1	0	0
1	0	1	1	0	1	0	0
1	0	1	1	1	1	0	1
1	1	0	0	0	1	0	0
1	1	0	0	1	1	0	1
1	1	0	1	0	1	0	1
1	1	0	1	1	1	1	0
1	1	1	0	0	1	0	1
1	1	1	0	1	1	1	0
1	1	1	1	0	1	1	0
1	1	1	1	1	1	1	1

2-Bit Adder cont.

- Equations are very complex, requiring 23 terms with 80 literals
- A two-level solution would require a 12-input gate for ***s1***
- Another problem that we would encounter in that implementation in the real world
    - There is limitation on number of inputs (called *fan-in)* for a gate
- Gates with 12 inputs may not be practical or may encounter delays of greater than Δ
- c_{out} can be implemented with two-level logic (with maximum fan-in of 7)

$$s_1 = c'_{in}a'_1b_1 + c'_{in}a_1b'_1 + c_{in}a'_1b'_1 + c_{in}a_1b_1 \qquad c_{out} = a_2b_2 + a_1b_1a_2 + a_1b_1b_2 + c_{in}b_1b_2 + c_{in}b_1a_2 + c_{in}a_1b_2 + c_{in}a_1a_2$$

4-Bit Adder

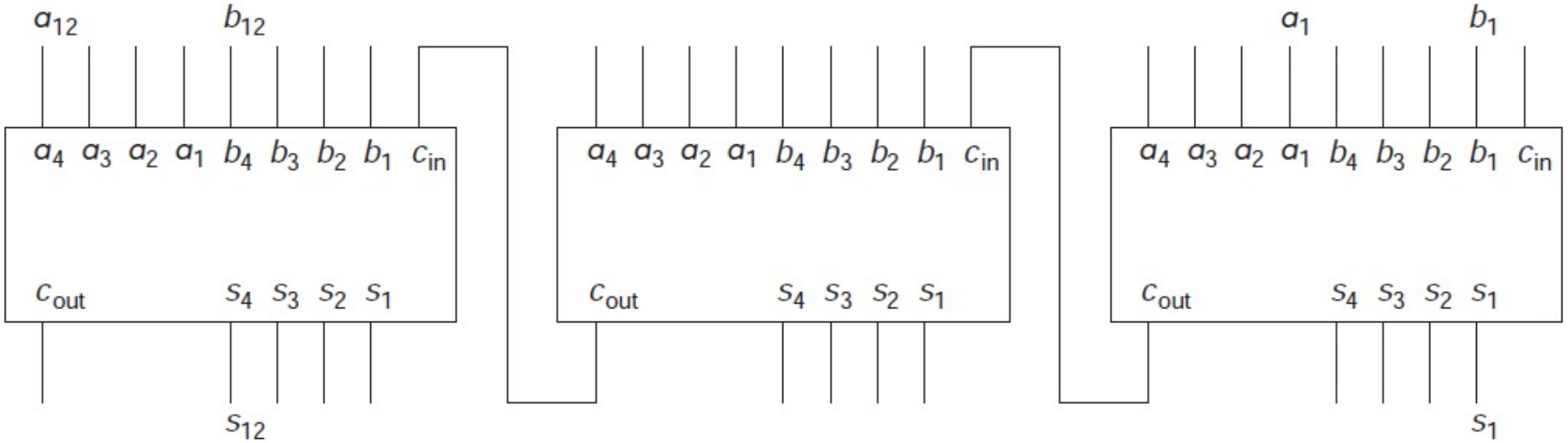
- Commercially 4-bit adders are available: 7483, 7483A, and 74283
  - Each is implemented differently, with three-level circuit for the carry-out
  - 7483A and 74283 differ only in pin connections
  - Each produces *sum* with four-level circuit, using mixture of NAND, NOR, AND, NOT, and Exclusive-OR gates

- Delay from *carry-in* to *carry-out* is 3Δ for each four bits
  - Producing a total delay of (3/4 n + 1)Δ (an extra delay for the last *sum*)
- 7483 ripples the carry internally (although it has a three-level chip *carry- out*);
  - It uses an eight-level circuit for *s*4

Larger Adders

- When larger adders are needed, 4-bit adders can be *cascaded*
- For example, a 12-bit adder, using three 4-bit adders (such as 7483) is shown in Figure 5.4, where each block represents a 4-bit adder

Figure 5.4 Cascading 4-bit adders.



Subtracors and Adder/Subtractor

- To do subtraction, we could develop the truth table for a 1-bit full subtractor (See Lab 6 Notes) and cascade as many of these as are needed, producing a borrow-ripple subtractor

- Most of the time, when subtractor is needed, an adder is needed as well
  - Because when we have to perform *M* – *N*, we add the 1's complement of subtrahend *N* to the minuend *M* and add 1 in the result (i.e. add 2's complement of *N* to *M*)

Subtracors and Adder/Subtractor

To build such an adder/subtractor, we need a signal line that is 0 for addition and 1 for subtraction

– Call that ***a'/s*** (short for *add'/subtract*)

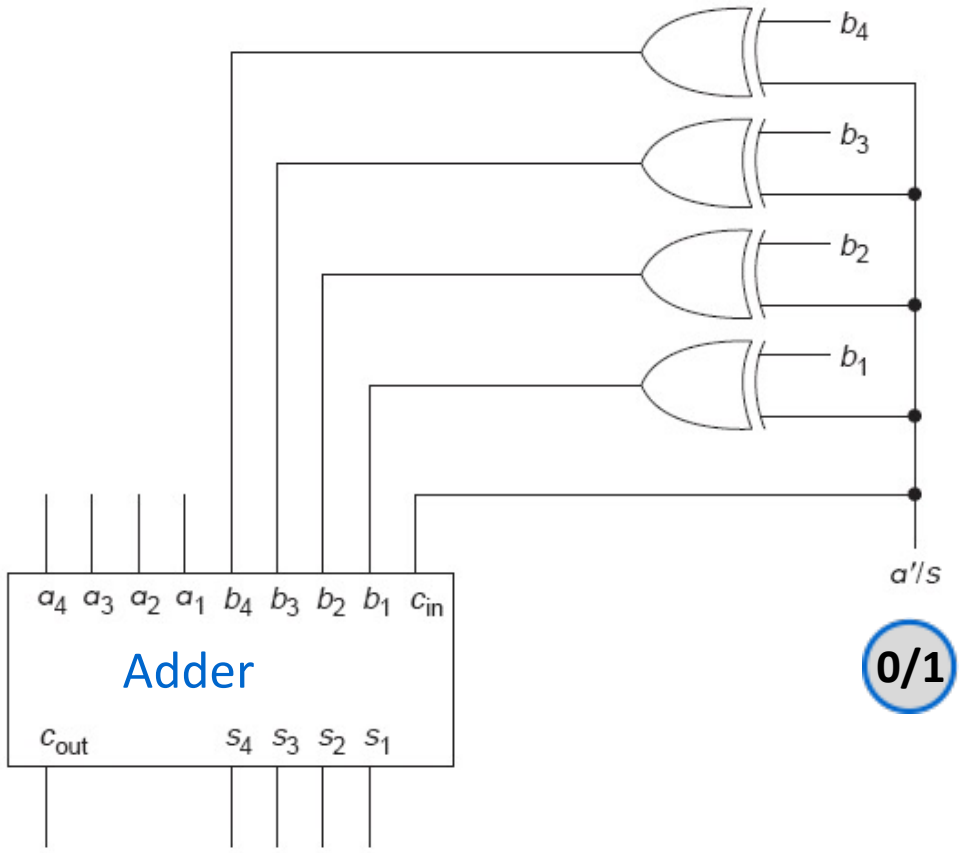
Also we need following two properties:

– $1 \oplus x = x'$

– $0 \oplus x = x$

Circuit for Adder/Subtractor is shown

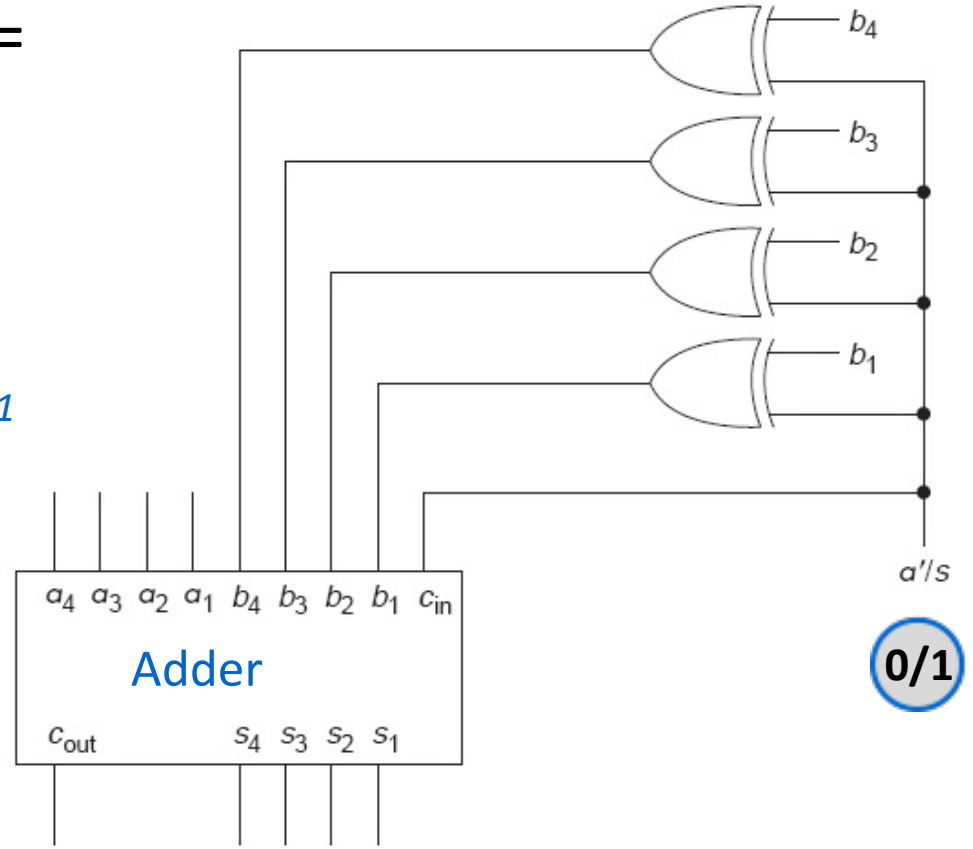
Figure 5.5 A 4-bit adder/subtractor.



Explanation

- When a'/s = 1, circuit is a subtractor, with c_{in} = 1 (used to find the 2'sC of *subtrahend* after taking its 1'sC)
- e.g.,
  - If *minuend* $a_4a_3a_2a_1$ = 1011, *subtrahend* $b_4b_3b_2b_1$ = 1010
  - By property $1 \oplus x = x'$
    - XORs output is 0101 (1'sC of subtrahend)
  - Adder will add *minuend* bits & *subtrahend* bits plus c_{in} (2'sC of subtrahend) producing *sum* $s_4s_3s_2s_1$ and end carry c_{out} (if any)

Figure 5.5 A 4-bit adder/subtractor.



Comparators

- Comparators are used to compare two numbers, producing an indication if they are equal or if one is larger than the other

- Exclusive-OR or Exclusive-NOR gates can be used in comparator circuits
  - XOR produces 1 if the two inputs are unequal and 0 otherwise
  - XNOR produces a 1 if the two inputs are equal and 0 otherwise

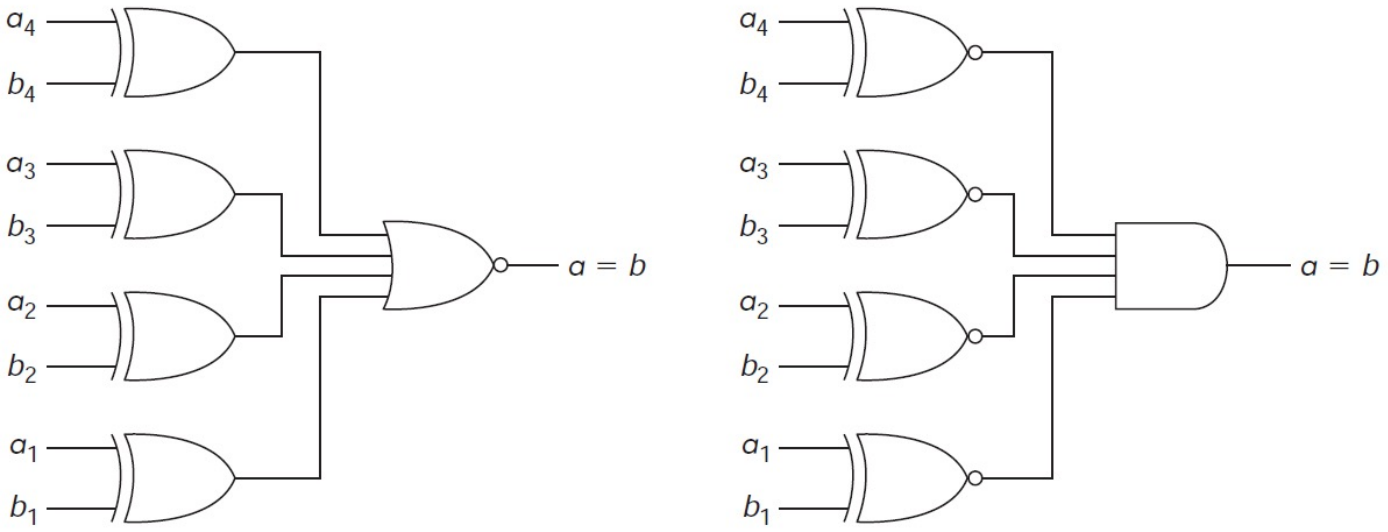
- Multibit numbers are unequal if any of the input pairs are unequal

4-Bit Comparators

A 4-bit comparator is shown in the following figure

- With XORs & NOR gate; output of NOR gate is 1 if the numbers are equal
- Also with XNORs & AND gate; output of AND gate is 1 if numbers are equal

Figure 5.6 Two 4-bit comparators.



(a) Exclusive-OR (b) Exclusive-NOR

n-Bit Comparators

- Comparators can be extended to any number of bits
- To build a 4-bit comparator that will indicate > and <, as well as = to (for unsigned numbers), we recognize that, starting at the MSB (a_4 and b_4)

$a > b$ if $a_4 > b_4$ or $(a_4 = b_4$ and $a_3 > b_3)$ or $(a_4 = b_4$ and $a_3 = b_3$ and $a_2 > b_2)$ or $(a_4 = b_4$ and $a_3 = b_3$ and $a_2 = b_2$ and $a_1 > b_1)$

$a < b$ if $a_4 < b_4$ or $(a_4 = b_4$ and $a_3 < b_3)$ or $(a_4 = b_4$ and $a_3 = b_3$ and $a_2 < b_2)$ or $(a_4 = b_4$ and $a_3 = b_3$ and $a_2 = b_2$ and $a_1 < b_1)$

$a = b$ if $a_4 = b_4$ and $a_3 = b_3$ and $a_2 = b_2$ and $a_1 = b_1$

Comparator Circuit

- A typical comparator circuit is shown

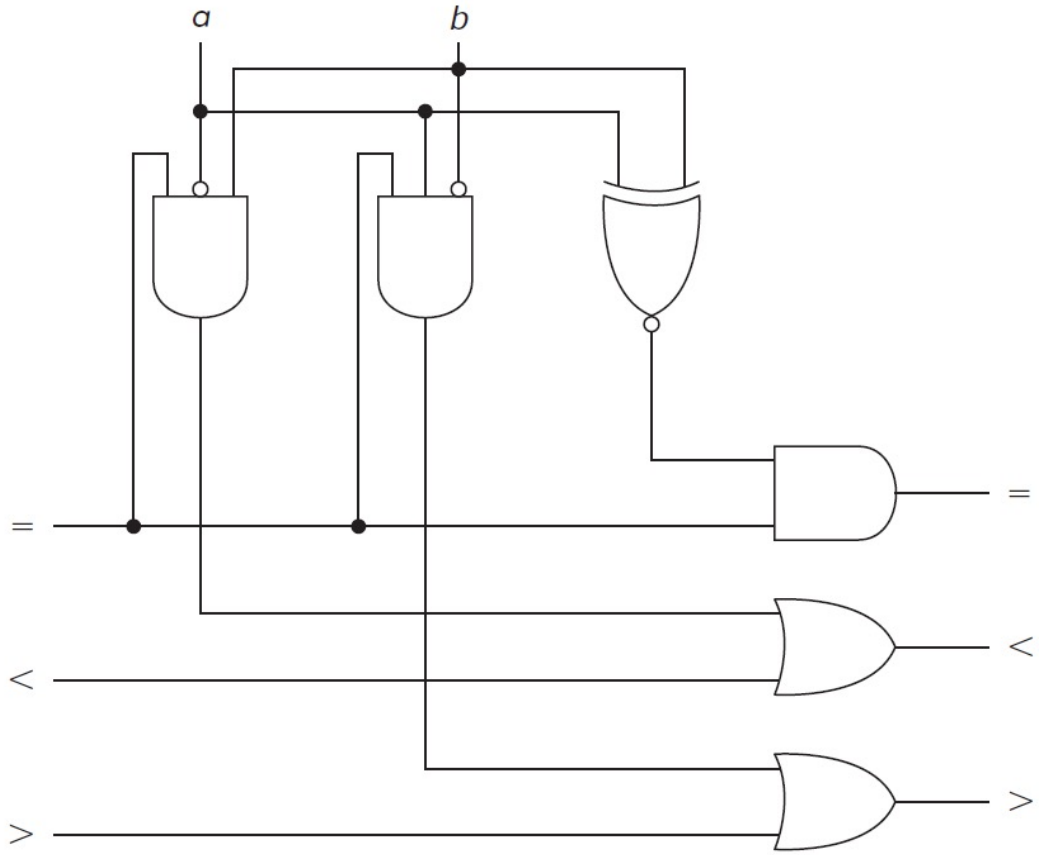
- To learn more about construction of 4-bit comparator circuit, see lab 6 notes

- Inputs

  = is 1

  while < and > are 0

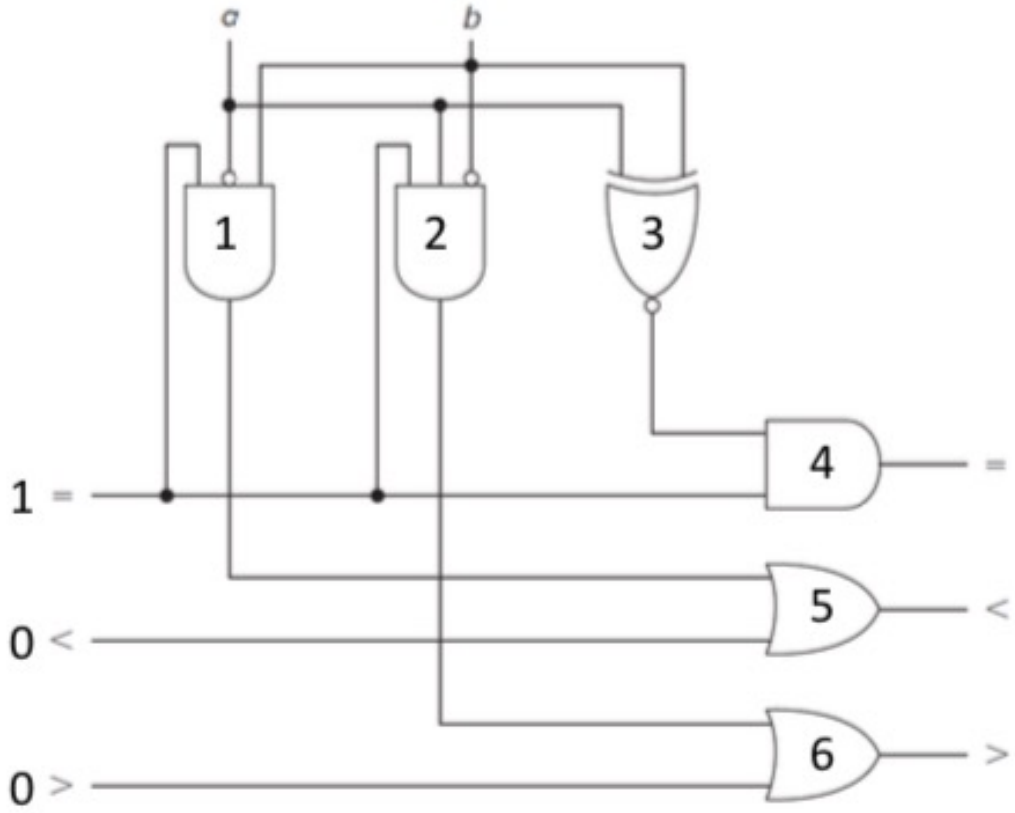
Figure 5.7 Typical bit of a comparator.



Explanation

- Suppose a = 0, b = 0 or a = 1, b = 1
- Then
  - Outputs of AND gates 1 and 2 will be 0 making outputs of OR gates 5 and 6 as 0
  - Output of XNOR gate no.3 will be 1
  - Since input = is also 1, output of AND gate 4 will also be 1, indicating a = b

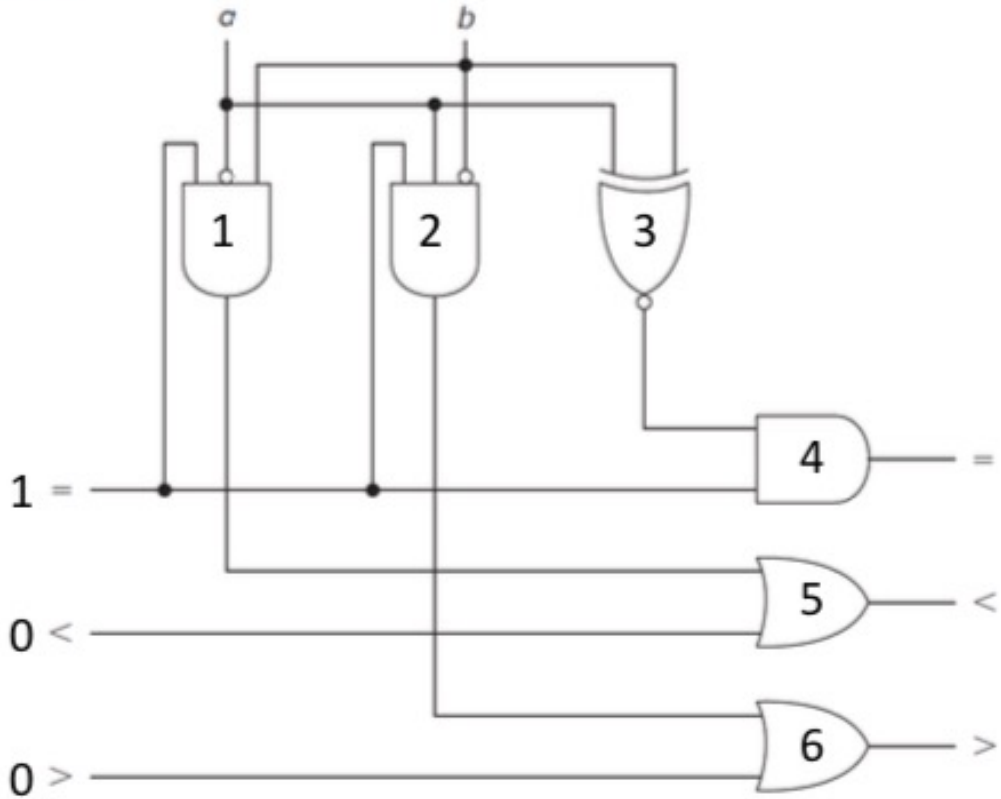
Figure 5.7 Typical bit of a comparator.



Explanation

- Suppose a = 0, b = 1
- Then
  - Output of AND gate no. 1 will be 1 producing output of OR gate no. 5 as 1 indicating a < b
  - Outputs of AND gate no. 4 and OR gate no. 6 will be 0

Figure 5.7 Typical bit of a comparator.



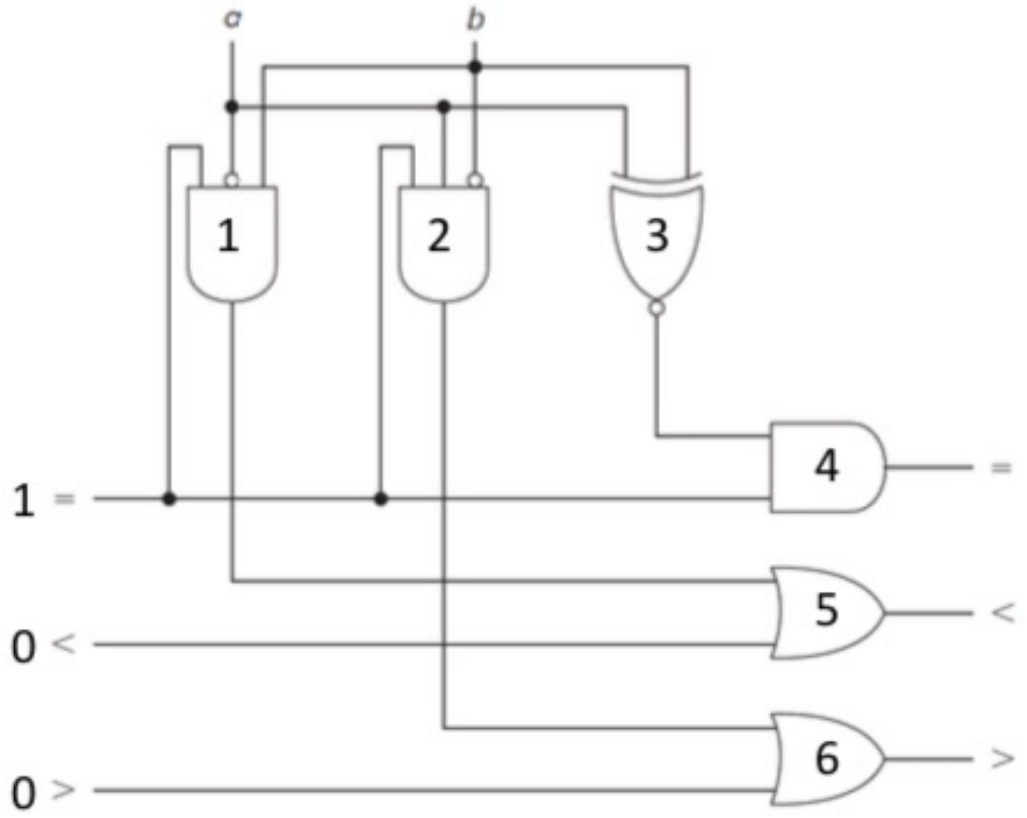
Explanation

Suppose a = 1, b = 0

Then

- Output of AND gate no. 2 will be 1 producing output of OR gate no. 6 as 1 indicating a > b
- Outputs of AND gate no. 4 and OR gate no. 5 will be 0

Figure 5.7 Typical bit of a comparator.



Binary Decoders

- A binary decoder is a device that, when activated, selects one of several output lines, based on a coded input signal
- Most commonly
  - Input is an *n*-bit binary number, and there are up to 2^n output lines
- TT for a two-input (four-output) decoder is shown in Table 5.2a

Table 5.2a An active high decoder.

a	*b*	*0*	*1*	*2*	*3*
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

2-Input Decoder

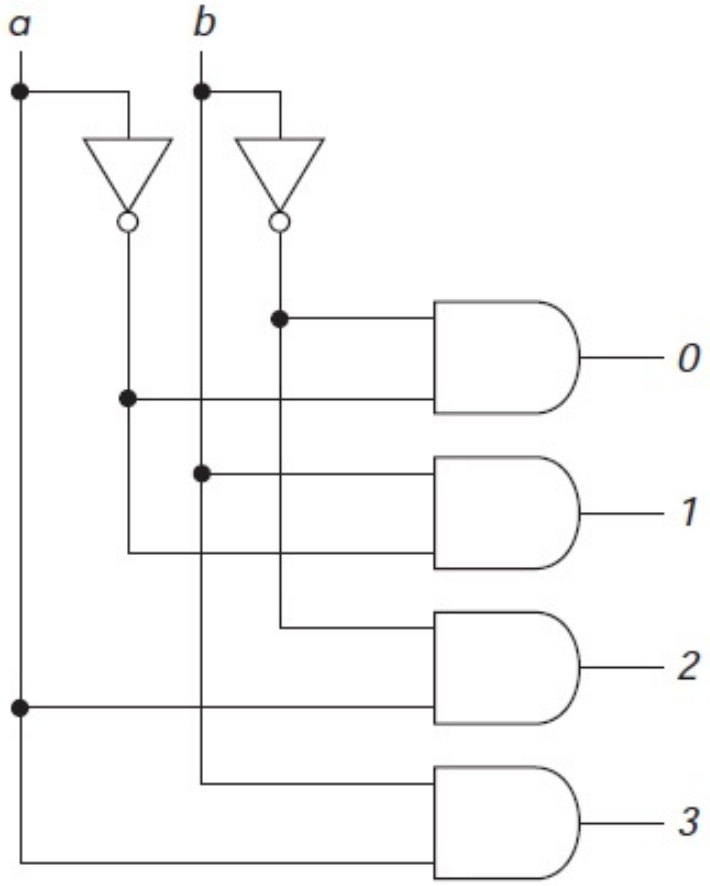
- Here the input is a binary number and output selected is the decimal equivalent of that binary number
- Output is *active high*
  - Active output is 1 and
  - Inactive ones are 0

Table 5.2a An active high decoder.

a	*b*	*0*	*1*	*2*	*3*
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Active-High Decoder

Figure 5.8a An active high decoder.



- Block diagram two-input (four-output) decoder is given in Figure 5.8a
  - Output
    - 0 is a'b'
    - 1 *is a'b*
    - 2 *is ab'*
    - 3 is *ab*

- Each output corresponds to one of the minterms for a two-variable function

Active-Low Decoder

- An active low output version of the decoder has one 0 corresponding to the input combination; the remaining outputs are 1

- TruthtabledescribingitisshowninTable5.2b

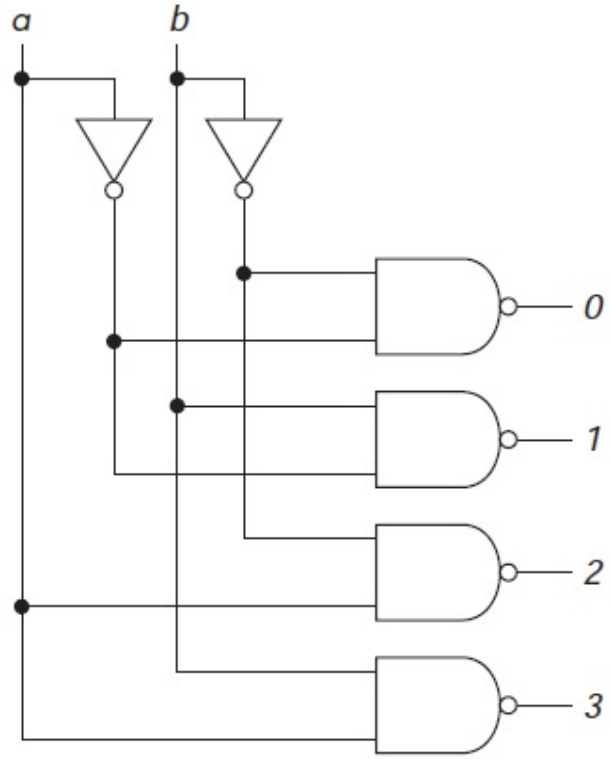
Table 5.2b An active low decoder.

a	*b*	*0*	*1*	*2*	*3*
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Active-Low Decoder

AND gates in the previous circuit are just replaced by NANDs

Figure 5.8b An active low decoder.



Decoder with Enable Input

- Most decoders also have one or more *enable* inputs

- Whensuchaninputis
  - *Active*, decoder behaves as described
  - *Inactive*, all of the outputs of the decoder are inactive

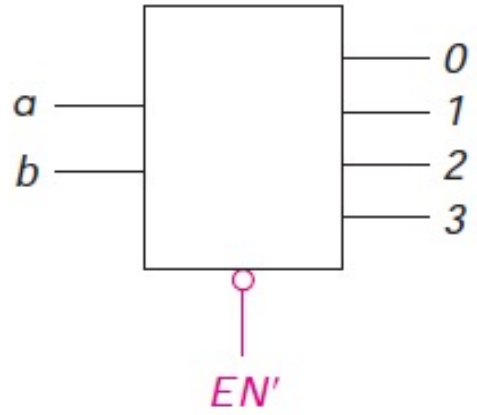
Decoder with Enable Input

- TT for an active high output (i.e. output is 1 when input is decoded) decoder with an active low enable input *EN'* (i.e. when enable is 0, it produces correct output) is shown

- Block diagram for such a decoder is also shown

Figure 5.9 Decoder with enable.

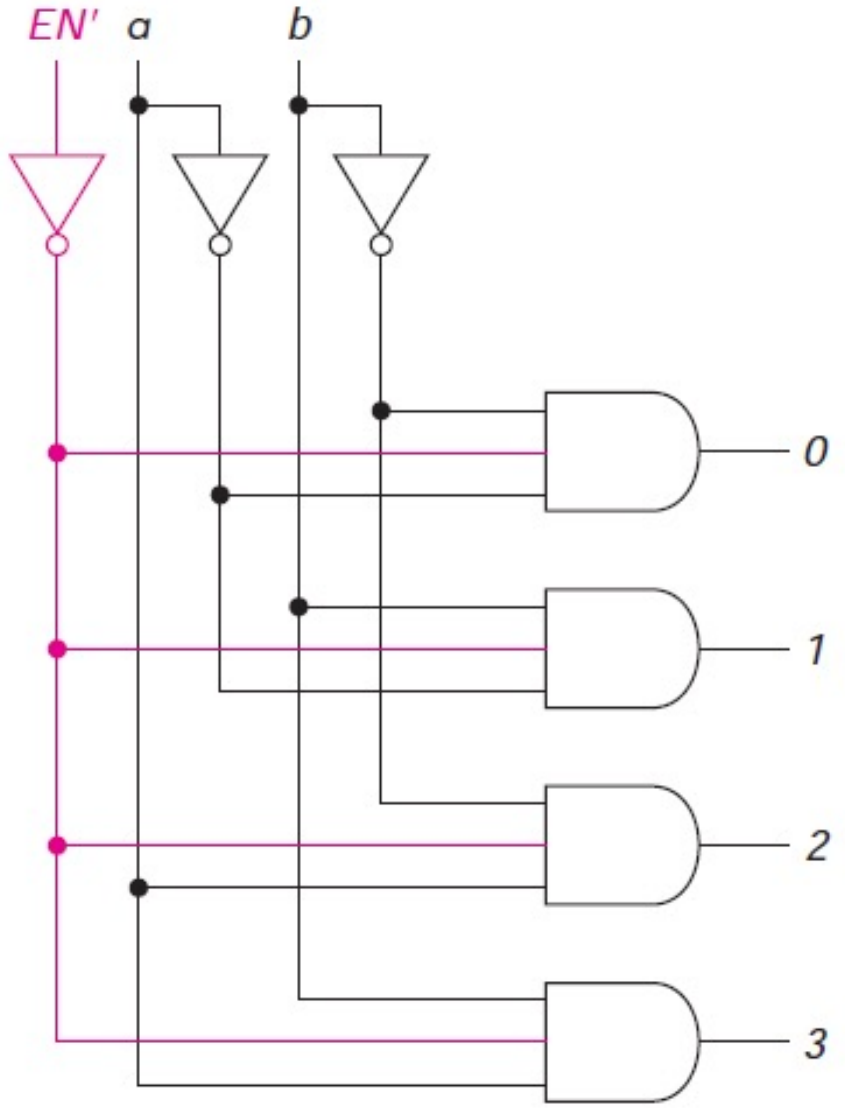
EN'	*a*	*b*	*0*	*1*	*2*	*3*
1	X	X	0	0	0	0
0	0	0	1	0	0	0
0	0	1	0	1	0	0
0	1	0	0	0	1	0
0	1	1	0	0	0	1



Note: Active low signals are often indicated with a circle (bubble), as shown

Decoder with Enable Input

The logic diagram of this decoder is shown

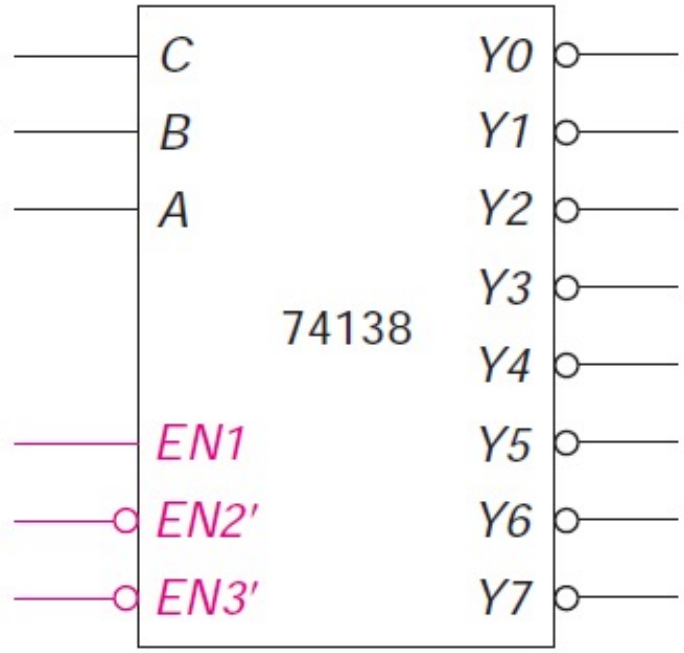


3-Inputs 8-Outputs Decoder

- A3-inputdecoderuses
  - 11 logic connections
    - 3 inputs and 8 outputs
  - in addition to two *power* connections
  - and one or more *enable* inputs

- Block diagram for the 74138 containing 1 of 8 decoder is shown

Figure 5.10 The 74138 decoder.

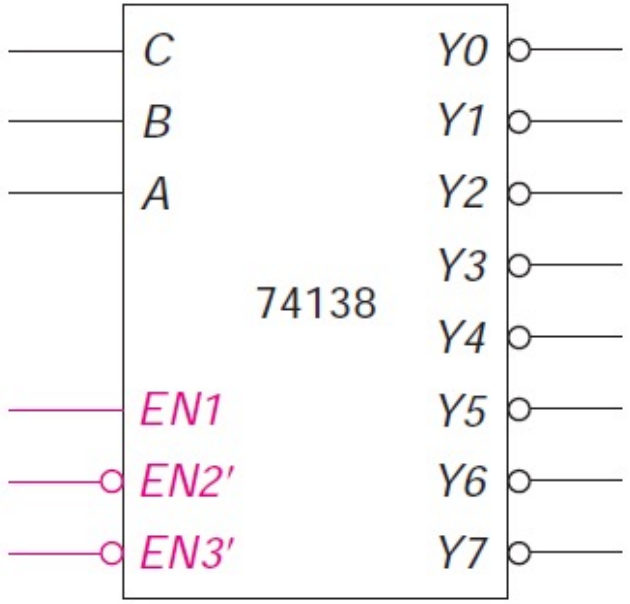


3-Inputs 8-Outputs Decoder

Chip(requiringa16-pinchip)with

- 2 for power connection
- *active low* outputs
- 3 *enable* inputs
  - One of which is *active high* (*EN1*), the other two (*EN2*, *EN3*) are *active low*
  - Only when all 3 *ENABLES* are active, i.e.,
    - *EN1 = 1, EN2' = 0,* and *EN3' = 0,* chip is *enabled*
  - Otherwise, all outputs are *inactive*, that is, 1

Figure 5.10 The 74138 decoder.



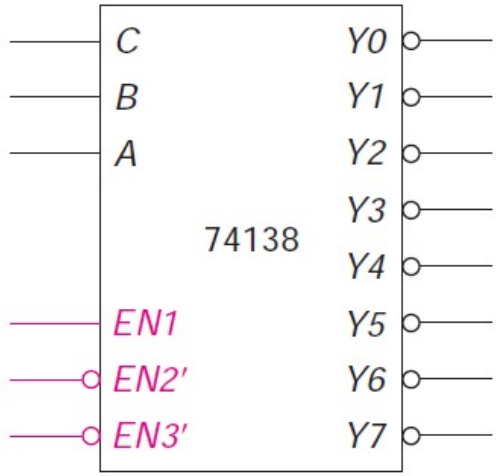
3-Inputs 8-Outputs Decoder

Truth table for the mentioned decoder is as follows:

Table 5.3 The 74138 decoder.

Enables			Inputs			Outputs							
EN1	*EN2'*	*EN3'*	*C*	*B*	*A*	*Y0*	*Y1*	*Y2*	*Y3*	*Y4*	*Y5*	*Y6*	*Y7*
0	X	X	X	X	X	1	1	1	1	1	1	1	1
X	1	X	X	X	X	1	1	1	1	1	1	1	1
X	X	1	X	X	X	1	1	1	1	1	1	1	1
1	0	0	0	0	0	0	1	1	1	1	1	1	1
1	0	0	0	0	1	1	0	1	1	1	1	1	1
1	0	0	0	1	0	1	1	0	1	1	1	1	1
1	0	0	0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	0	0	1	1	1	1	0	1	1	1
1	0	0	1	0	1	1	1	1	1	1	0	1	1
1	0	0	1	1	0	1	1	1	1	1	1	0	1
1	0	0	1	1	1	1	1	1	1	1	1	1	0

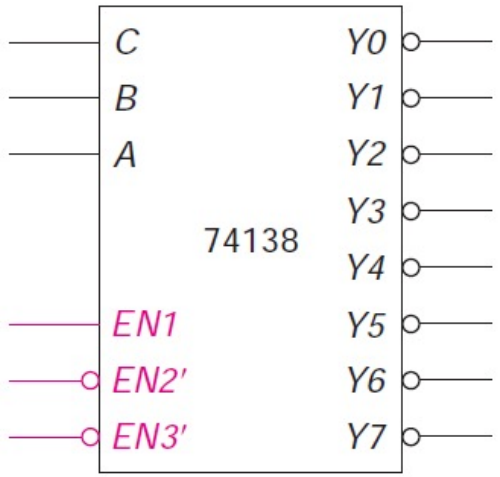
Figure 5.10 The 74138 decoder.



Important Note

- In the circuit (this is true in many of commercial IC packages) inputs are labeled *C, B, A* (with *C* the highest-order bit)
- In previous examples, we have made *A* the highest-order bit

Figure 5.10 The 74138 decoder.



Applications of Decoders

- One application of decoders is to select one of many devices, each of which has a unique address

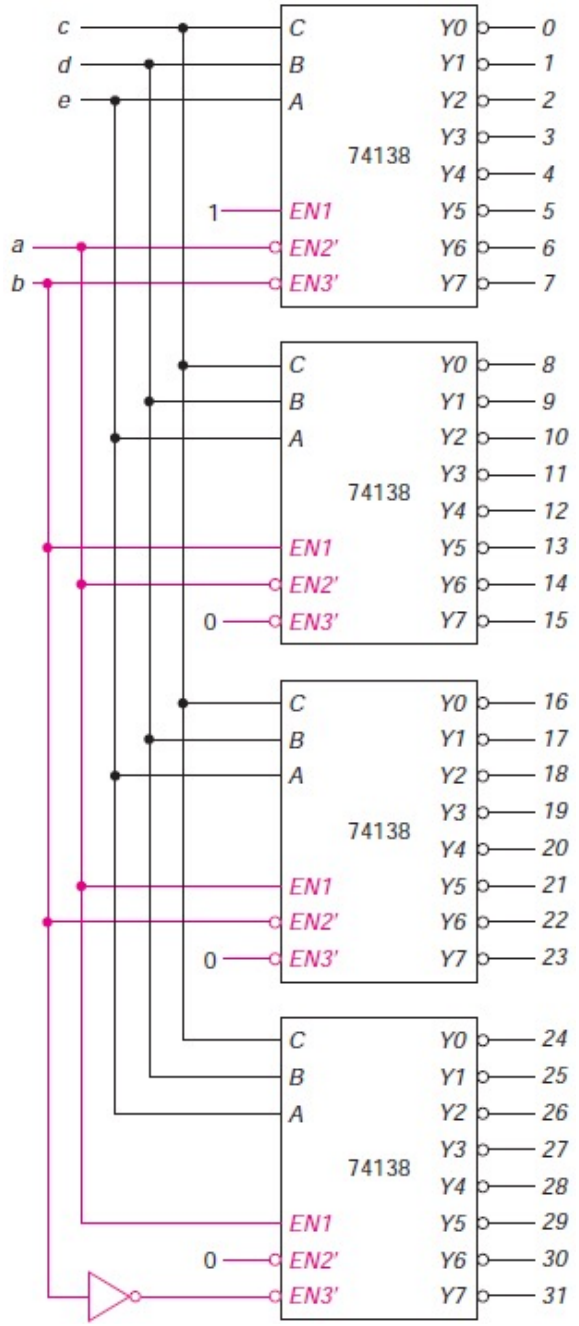
- Address is the input to the decoder
  - One output is active, to select the one device that was addressed

Example

- We have available 74138 decoders and wish to select one of 32 devices, present in this chip.

- We would need four such decoders
  - One of these would select one of the first eight addressed devices; another would select one of the next eight, and so forth

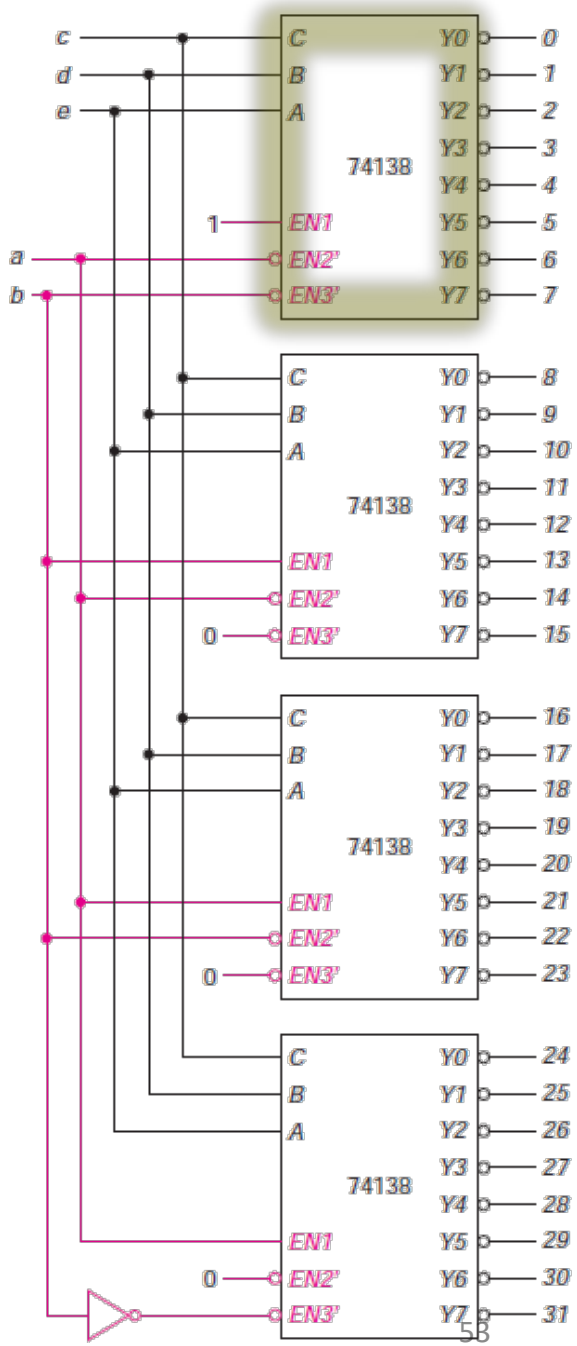
- If addresses were given by bits *a, b, c, d, e*, then
  - *c, d, e* would be the inputs (to *C, B, A* in order) for each of the four decoders, and
  - *a, b* would be used to enable the appropriate one



Example cont.

Thus EN1 = 1, EN2' = 0, and *EN3' = 0,* chip is *enabled*

- First decoder enabled when *a = b =* 0

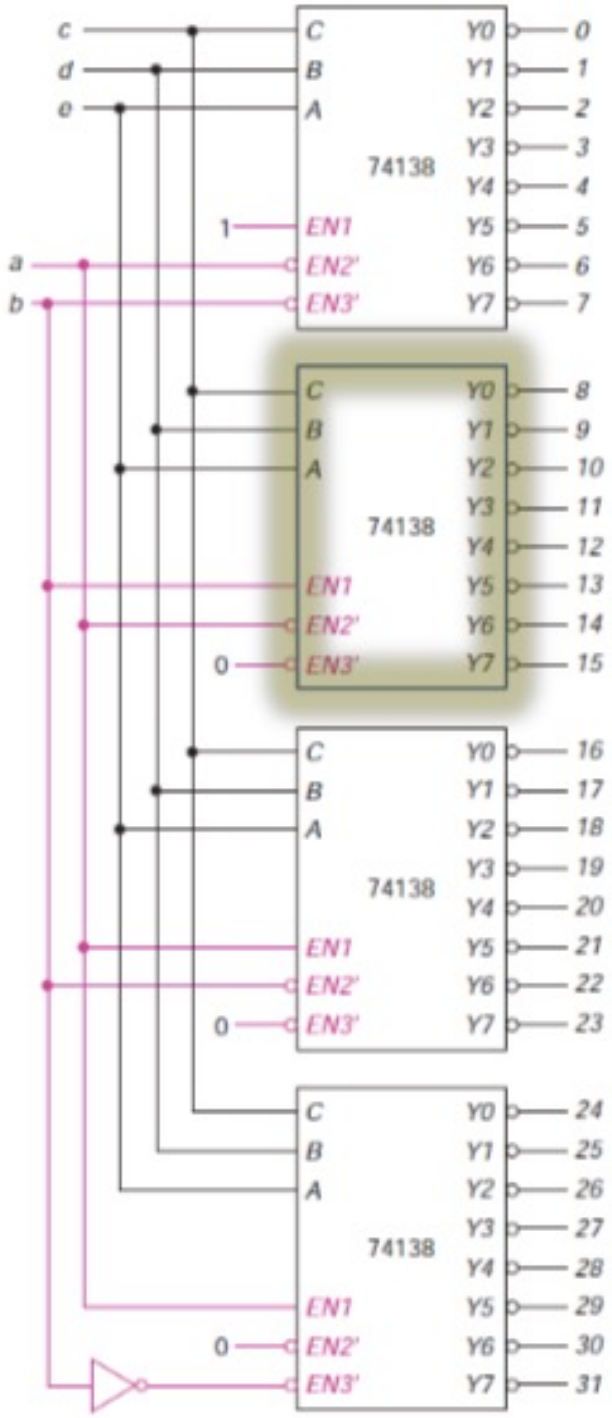


Example cont.

Thus

EN1 = *1*, *EN2'* = *0*, and *EN3'* = *0*, chip is *enabled*

- First decoder enabled when $a = b = 0$
- Second decoder enabled when $a = 0$ and $b = 1$

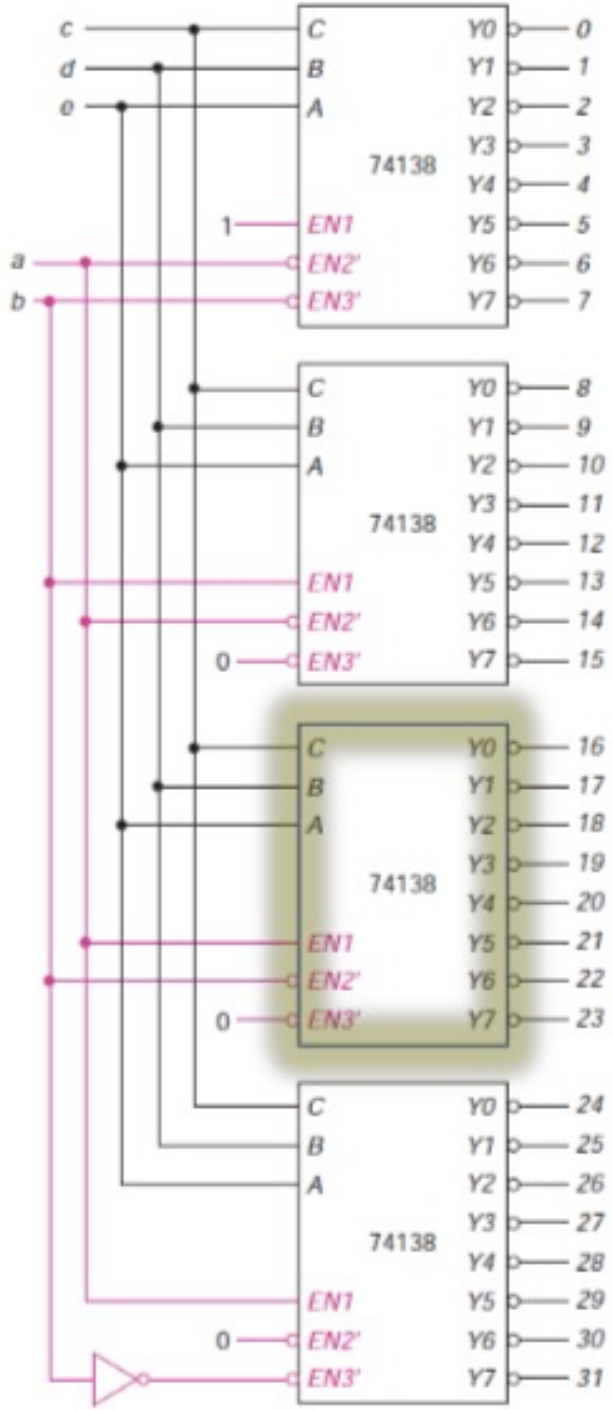


Example cont.

Thus

EN1 = 1, EN2' = 0, and *EN3' = 0,* chip is *enabled*

- First decoder enabled when $a = b = 0$
- Second decoder enabled when $a = 0$ and $b = 1$
- Third decoder enabled when $a = 1$ and $b = 0$



Example cont.

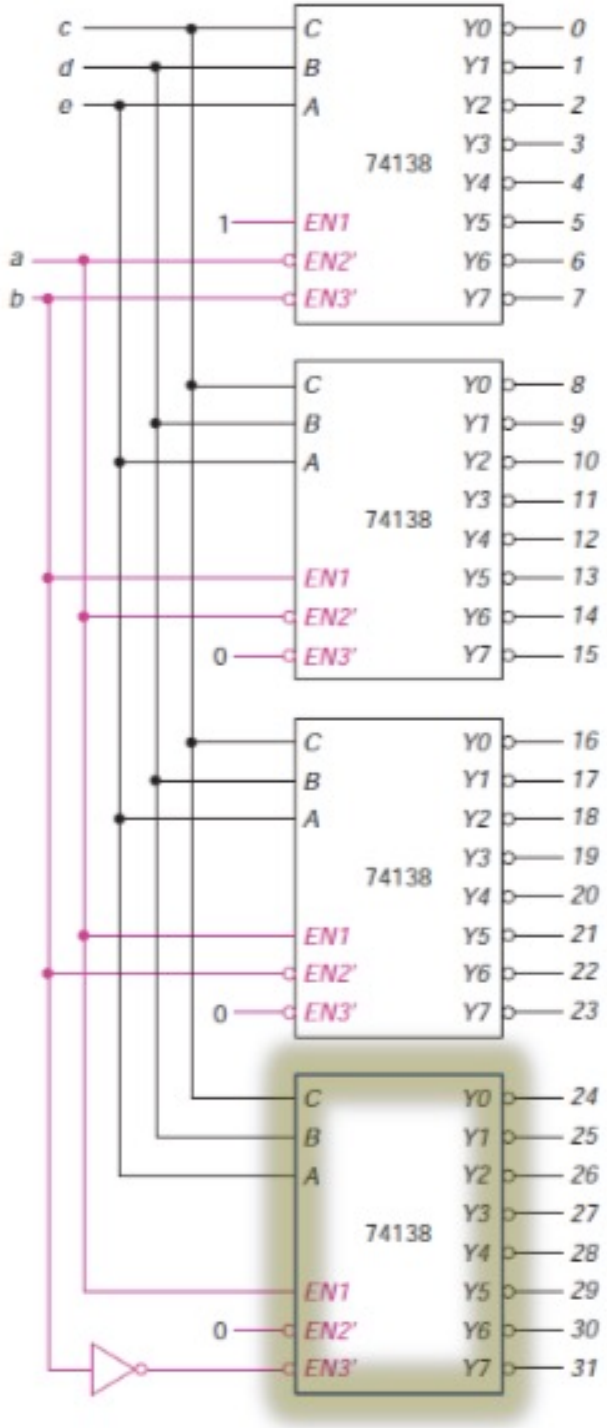
EN1 = 1, *EN2'* = 0, and *EN3'* = 0, chip is *enabled*

Thus

- First decoder enabled when $a = b = 0$
- Second decoder enabled when $a = 0$ and $b = 1$
- Third decoder enabled when $a = 1$ and $b = 0$
- Fourth decoder enabled when $a = b = 1$

Since we have two *active low enable* inputs and one *active high enable*

- Only the fourth decoder would require a NOT gate for the *enable* input assuming a' and b' are not available

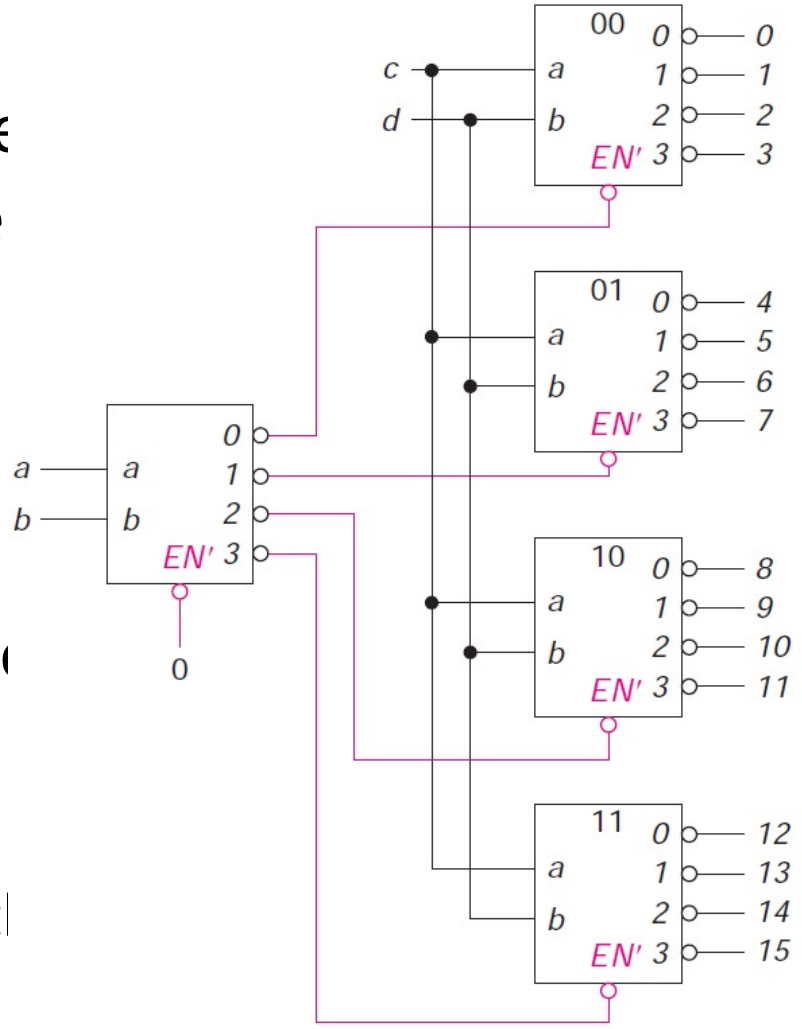


Applications of Decoders

Sometimes, an extra decoder is used to enable other decoders

Example

- We had a two-input, four-output active low decode with an active low enable, and needed to select one 16 devices
- We could use one decoder to choose among four groups of devices, based on two of inputs
- Most commonly, first two (highest order) inputs ar used, so
  - Groupings are devices 0–3, 4–7, 8–11, and 12–15
  - For each group, one decoder is used to choose among t four devices in that group



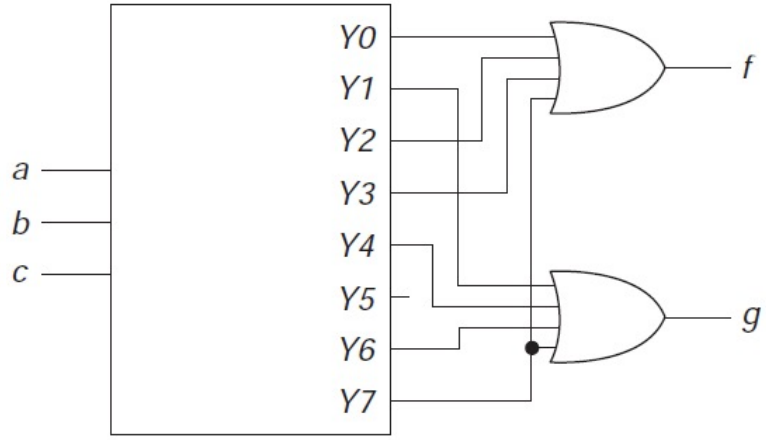
Applications of Decoders

- Another application of decoders is implementation of logic functions
- Each active high output of a decoder corresponds to a minterm of that function
- Thus, all we need is an OR gate connected to the appropriate outputs
- With an active low output decoder, the OR gate is replaced by a NAND (making a NAND-NAND circuit from an AND-OR circuit)

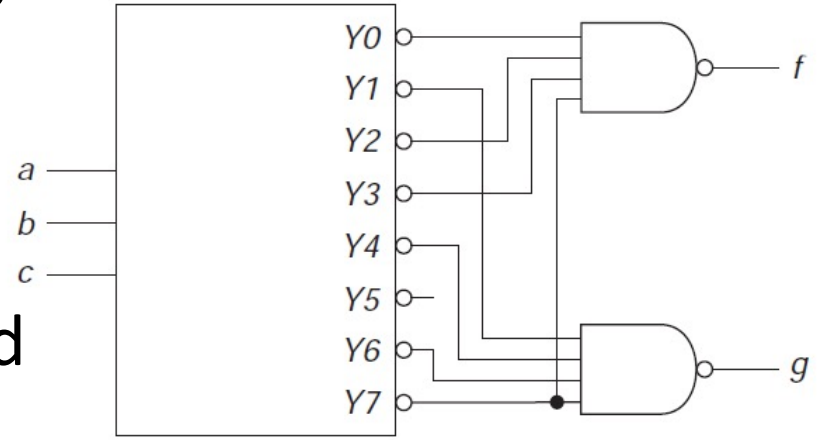
Example

- Two functions are given in sum of minterms as:

$$f(a, b, c) = \Sigma m(0, 2, 3, 7)$$
$$g(a, b, c) = \Sigma m(1, 4, 6, 7)$$



- To implement functions using decoders, only need to pass the corresponding outputs of the decoder through OR gate to produce relevant functions
  - Implementation of such circuit is shown, using
    - OR gates (with active high output decoder) and
    - NAND gates (with active low output decoder)



Example

- Suppose three functions *f*, *g*, and *h* are given as follows:

$$f(a, b, c, d) = \sum (0, 2, 3, 4, 7, 8, 10, 11, 15)$$
$$g(a, b, c, d) = \sum (0, 1, 2, 5, 8, 9, 10, 12, 13, 15)$$
$$h(a, b, c, d) = \sum (2, 4, 6, 8, 10, 12, 14, 15)$$

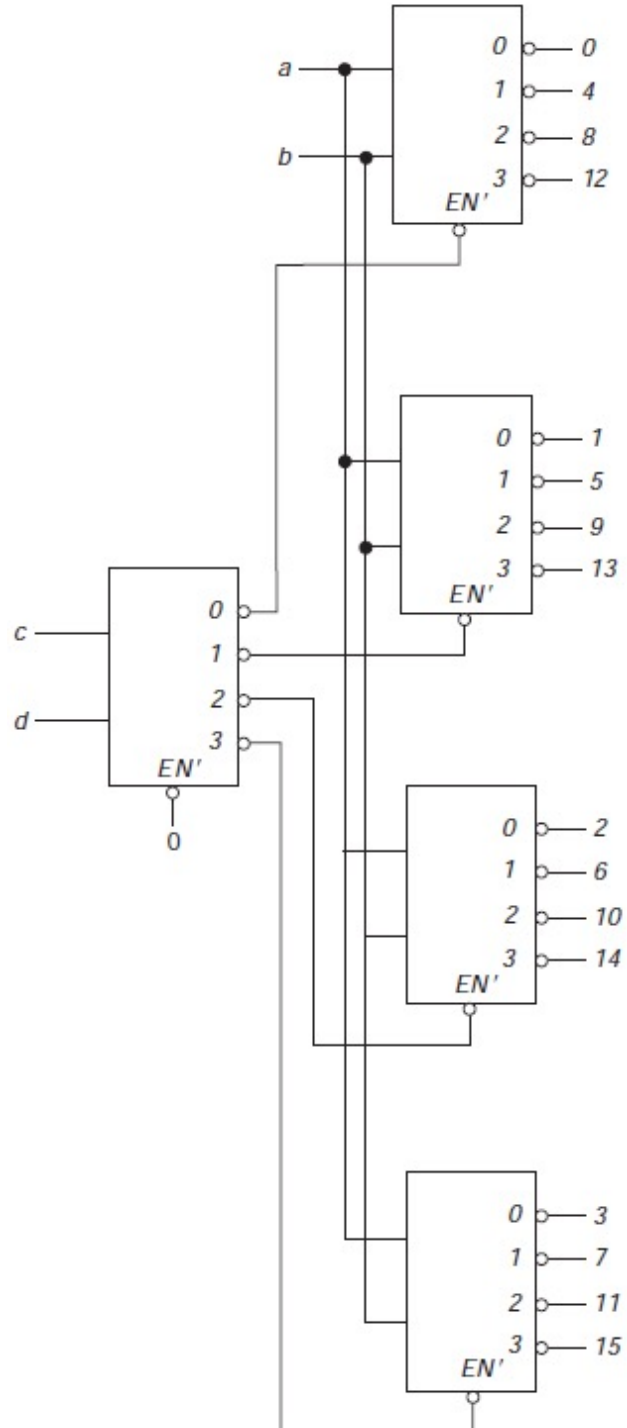
- We want to implement these functions using only the decoder defined by the following truth table and three NAND gates.

EN'	*a*	*b*	*0*	*1*	*2*	*3*
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

Example cont.

- From truth table, we see that it is an active low output decoder with active low enable input
- By using *c* and *d* for the inputs to the first decoder, and *a* and *b* for the inputs to the second level decoders, we get the shown layout

EN'	*a*	*b*	*0*	*1*	*2*	*3*
1	X	X	1	1	1	1
0	0	0	0	1	1	1
0	0	1	1	0	1	1
0	1	0	1	1	0	1
0	1	1	1	1	1	0

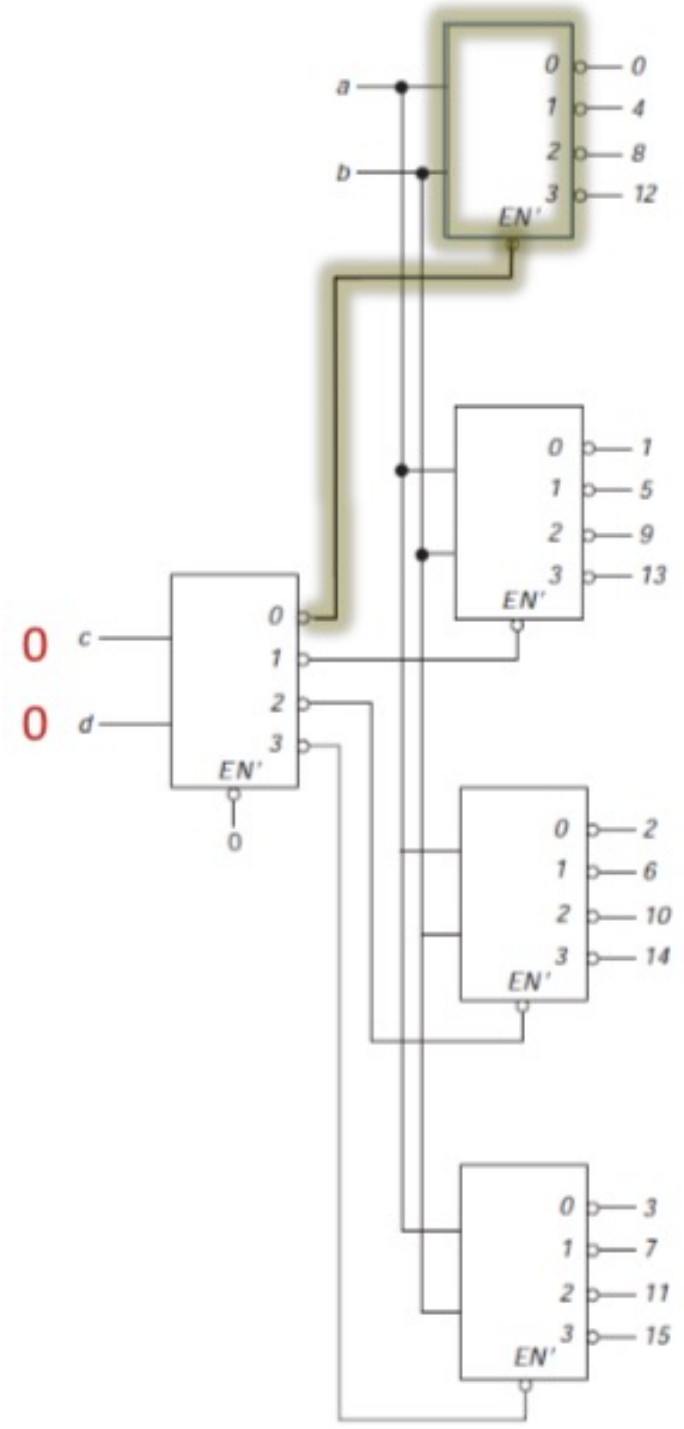


Example cont.

- When *c* = 0 and *d* = 0, output of first level decoder will be at 0
  - will enable the first of second level decoders

- So, when we change the values of *a* and *b* as 00, 01, 10, 11, we get the outputs as 0, 4, 8 and 12
  - Because bit combinations for *abcd* will become

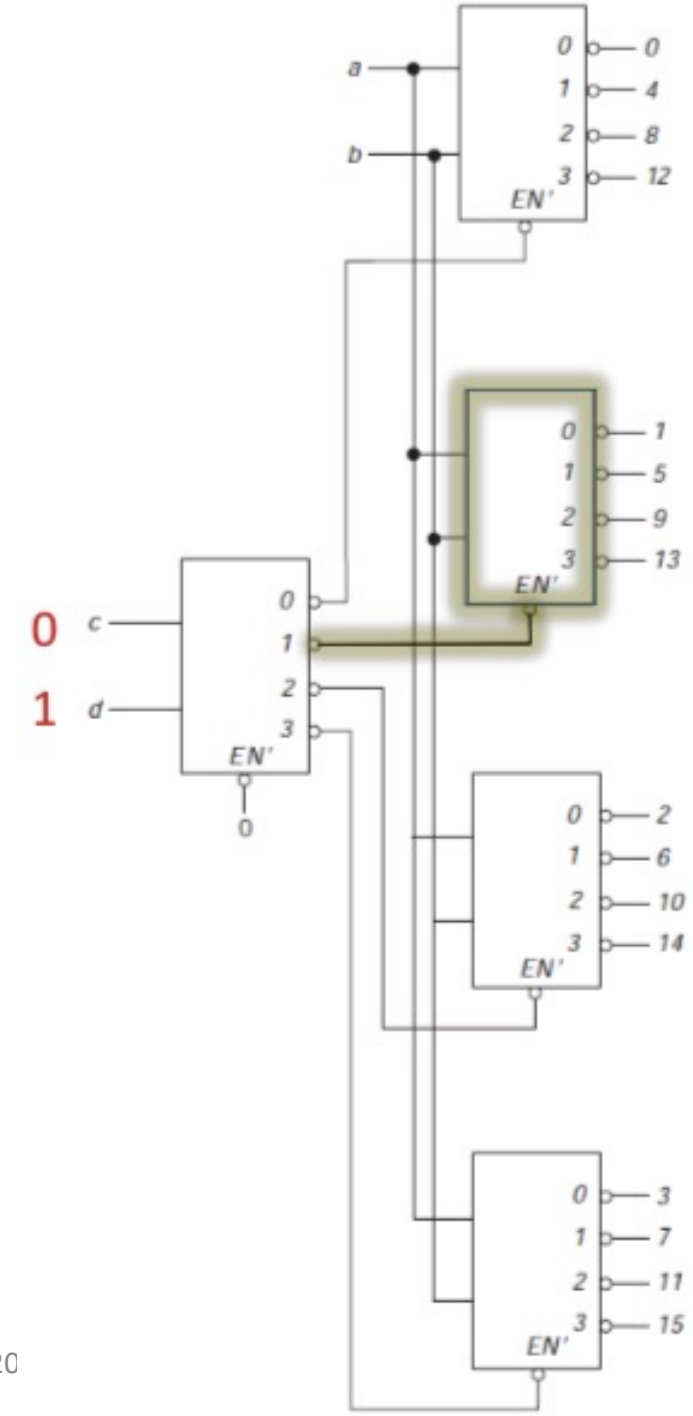
    0000, 0100, 1000, and 1100 respectively



Example cont.

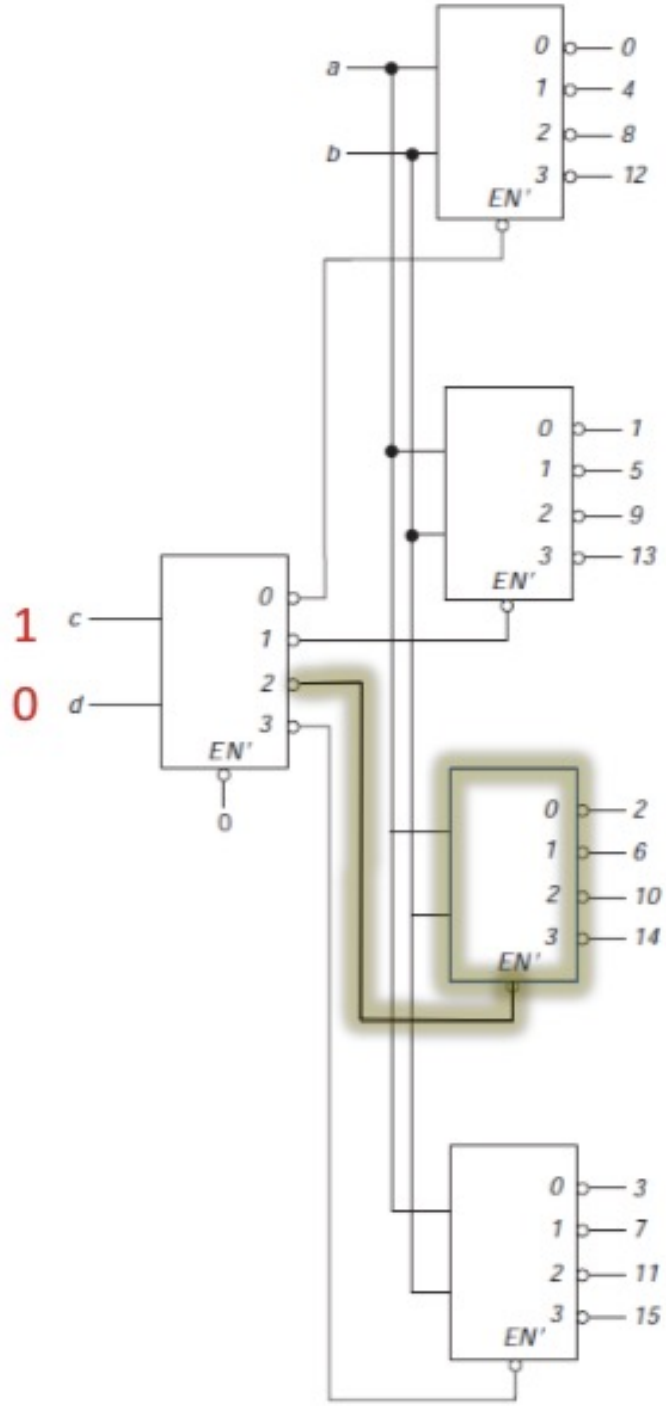
- When *c* = 0 and *d* = 1, output of first level decoder will be at 1,
  - will enable the second of second level decoders

- So, when we change the values of *a* and *b* as 00, 01, 10, 11, we get the outputs as 1, 5, 9 and 13
  - Because bit combinations for *abcd* will become 0001, 0101, 1001, and 1101 respectively



Example cont.

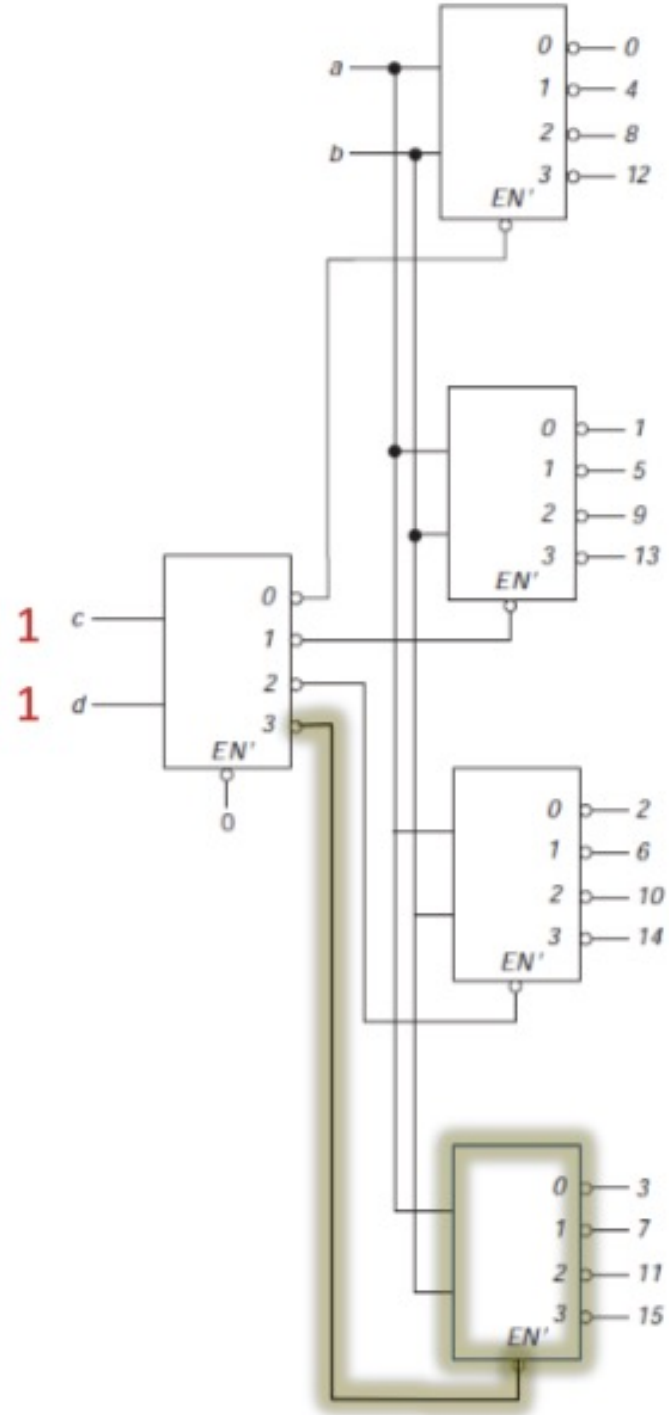
- When *c* = 1 and *d* = 0, output of first level decoder will be at 2
  - will enable the third of second level decoders
- So, when we change the values of *a* and *b* as 00, 01, 10, 11, we get the outputs as 2, 6, 10 and 14
  - Because bit combinations for *abcd* will become 0010, 0110, 1010, and 1110 respectively



Example cont.

- When *c* = 1 and *d* = 1, output of first level decoder will be at 3
  - will enable the fourth of second level decoders

- So, when we change the values of *a* and *b* as 00, 01, 10, 11, we get the outputs as 3, 7, 11 and 15
  - Because bit combinations for *abcd* will become 0011, 0111, 1011, and 1111 respectively



Example cont.

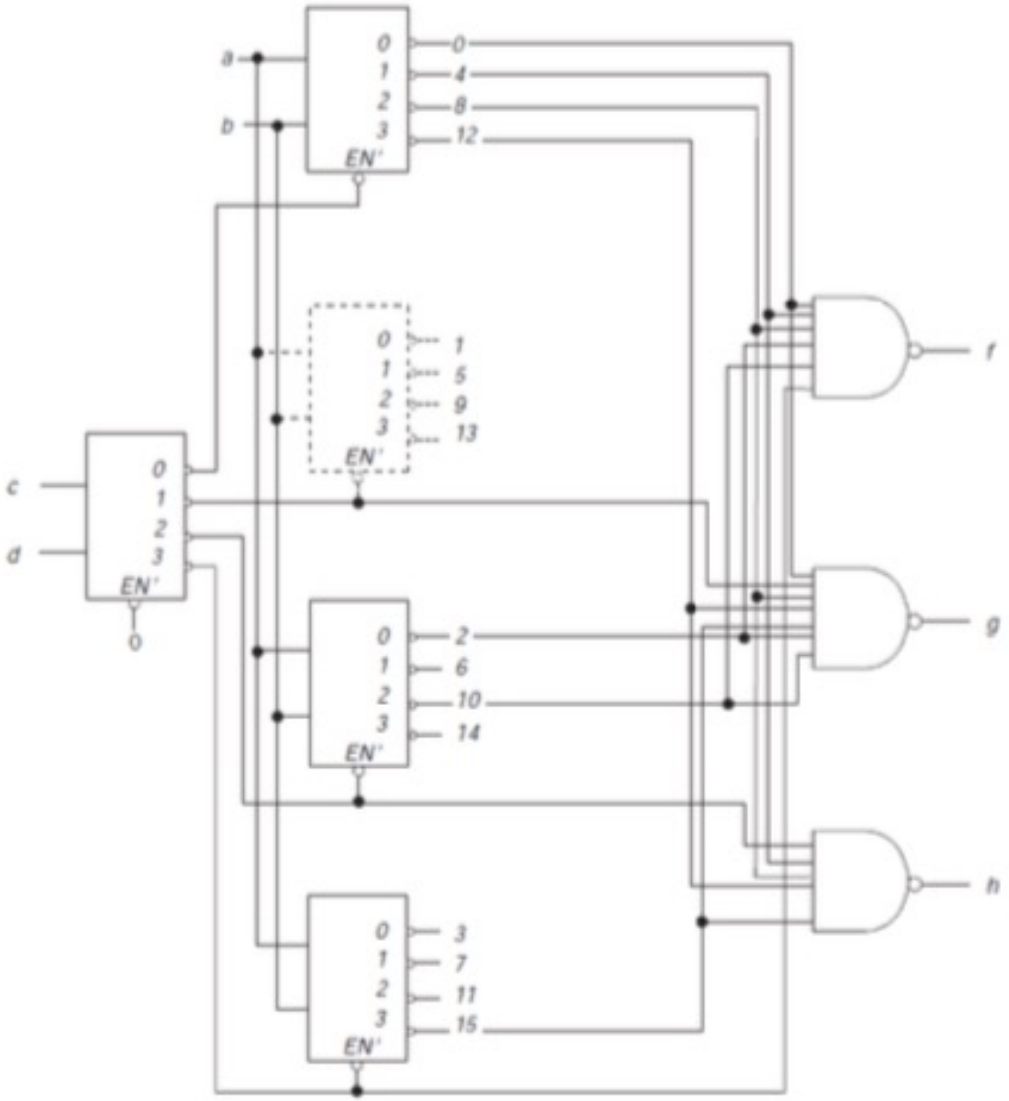
- If we use previous layout, we would need
  – 9-input NAND for *f*, 10-input gate for *g*, 8-input gate for *h*
- Using *c* and *d* for inputs to first decoder
  – This would still require same set of NAND gates if we did not take
- advantage that for *f*, minterms 3, 7, 11, and 15 are all included
  • Thus, we can connect *enable* input to fourth decoder directly to *f* output gate
- Same approach can be taken for *g* and *h*

$$f(a, b, c, d) = \sum (0, 2, 3, 4, 7, 8, 10, 11, 15)$$
$$g(a, b, c, d) = \sum (0, 1, 2, 5, 8, 9, 10, 12, 13, 15)$$
$$h(a, b, c, d) = \sum (2, 4, 6, 8, 10, 12, 14, 15)$$

Example cont.

- All NAND gates now have less than 8 inputs
- Also, one of the decoders is not necessary (shown in dashed lines), since its outputs are not used
  - Because the 01 column of the maps either have no 1's or all 1's

$f = \sum (0, 2, 3, 4, 7, 8, 10, 11, 15)$
$g = \sum (0, 1, 2, 5, 8, 9, 10, 12, 13, 15)$
$h = \sum (2, 4, 6, 8, 10, 12, 14, 15)$

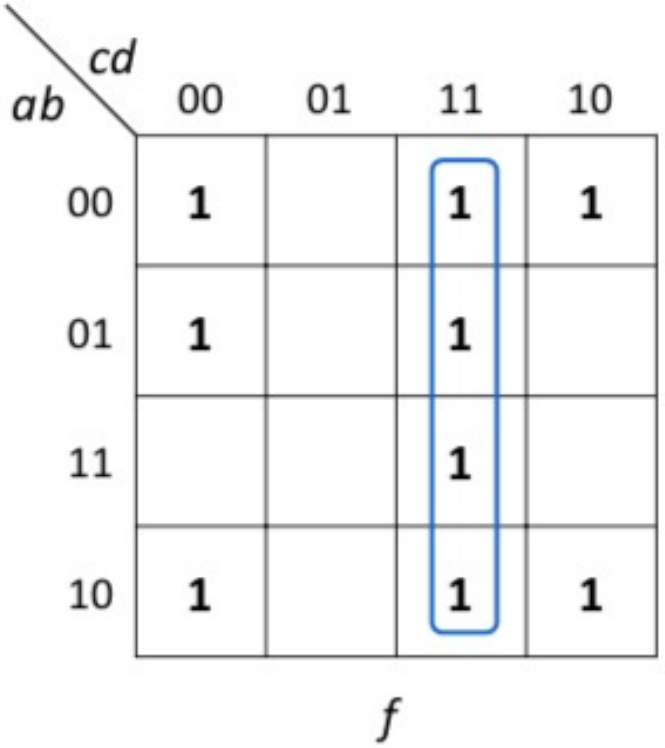


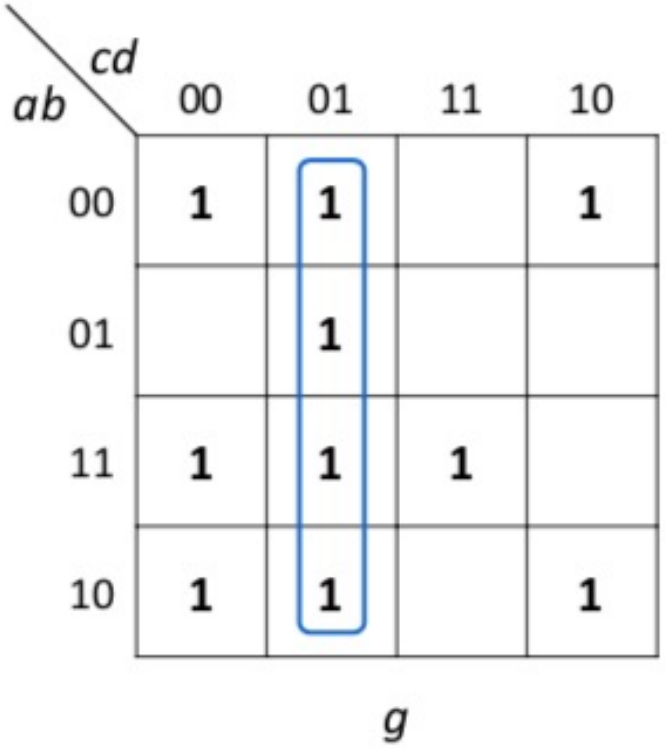
Example cont.

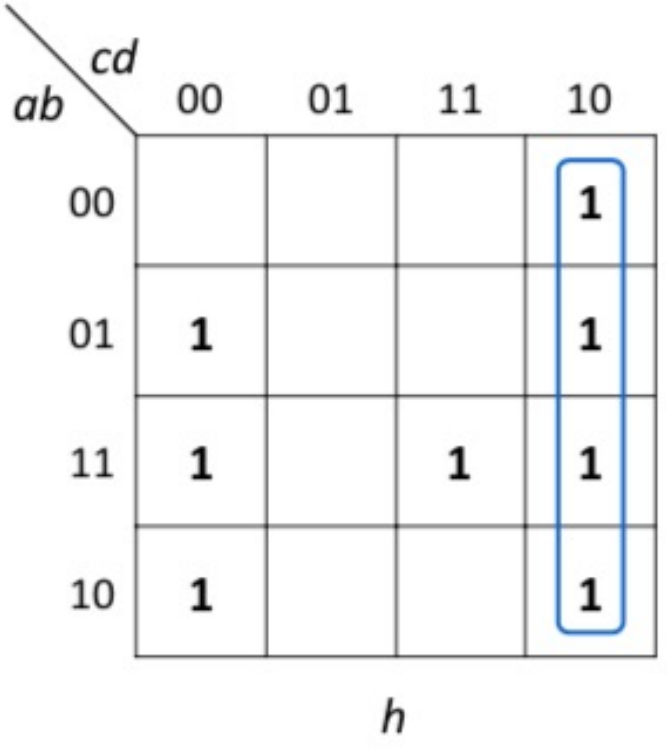
- If we have a restriction to use less than 8-input NAND gates, we cannot pass the required outputs of all decoders through the NAND gates, which correspond to the minterms present in the given functions *f*, *g* and *h*
- For this, we draw the K-Maps for *f*, *g* and *h* and get:

$$f(a, b, c, d) = \sum (0, 2, 3, 4, 7, 8, 10, 11, 15)$$
$$g(a, b, c, d) = \sum (0, 1, 2, 5, 8, 9, 10, 12, 13, 15)$$
$$h(a, b, c, d) = \sum (2, 4, 6, 8, 10, 12, 14, 15)$$

Example cont.







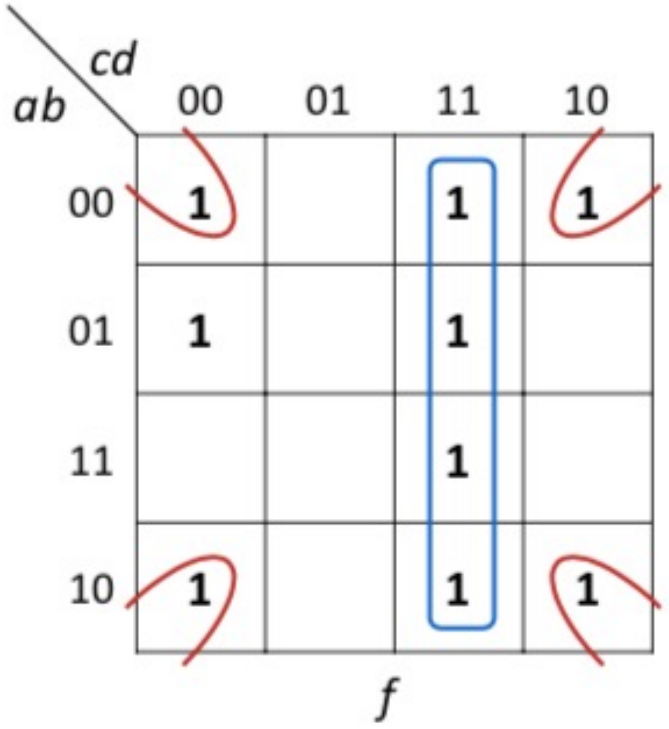
$f(a, b, c, d) = \sum (0, 2, 3, 4, 7, 8, 10, 11, 15)$
$g(a, b, c, d) = \sum (0, 1, 2, 5, 8, 9, 10, 12, 13, 15)$
$h(a, b, c, d) = \sum (2, 4, 6, 8, 10, 12, 14, 15)$

Example cont.

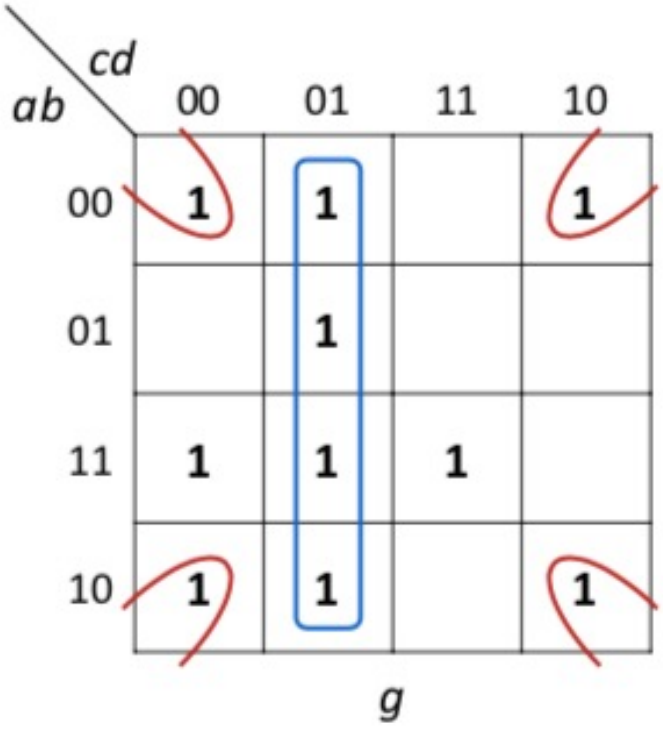
- So, when we make group of four 1's in each of these maps, we get
  - *cd* for *f*
  - *c'd* for *g*
  - *cd'* for *h*
- Hence, we can pass
  - output 3 (as *c*=1 and *d*=1) of first level decoder directly to NAND gate of *f*
  - output 1 (as *c*=0 and *d*=1) of first level decoder directly to NAND gate of *g*
  - output 2 (as *c*=1 and *d*=0) of first level decoder directly to NAND gate of *h*

Example cont.

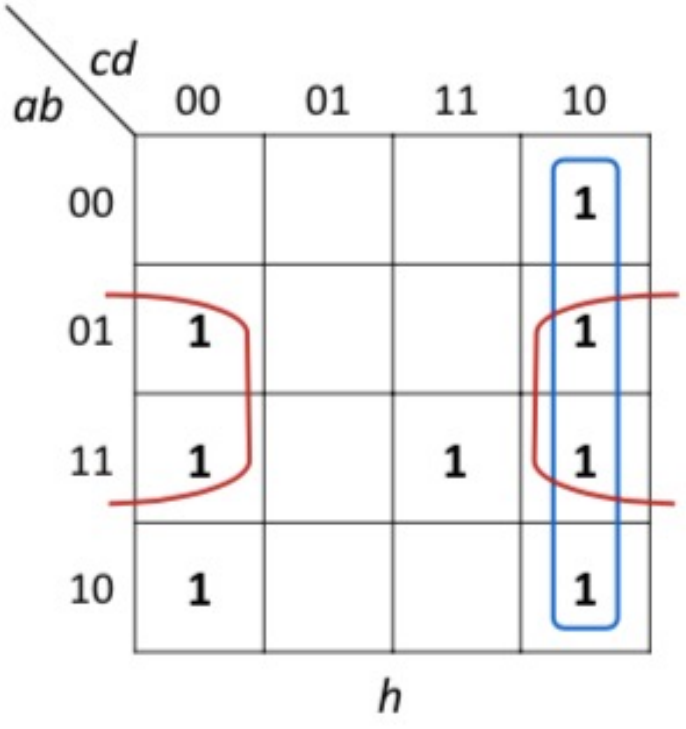
- To implement this with only 4-input NAND gates, we must take a different approach; next set of maps shows several groups of four



$f = a'bc'd' + cd + b'd'$



$g = c'd + b'd' + abc'd' + abcd$



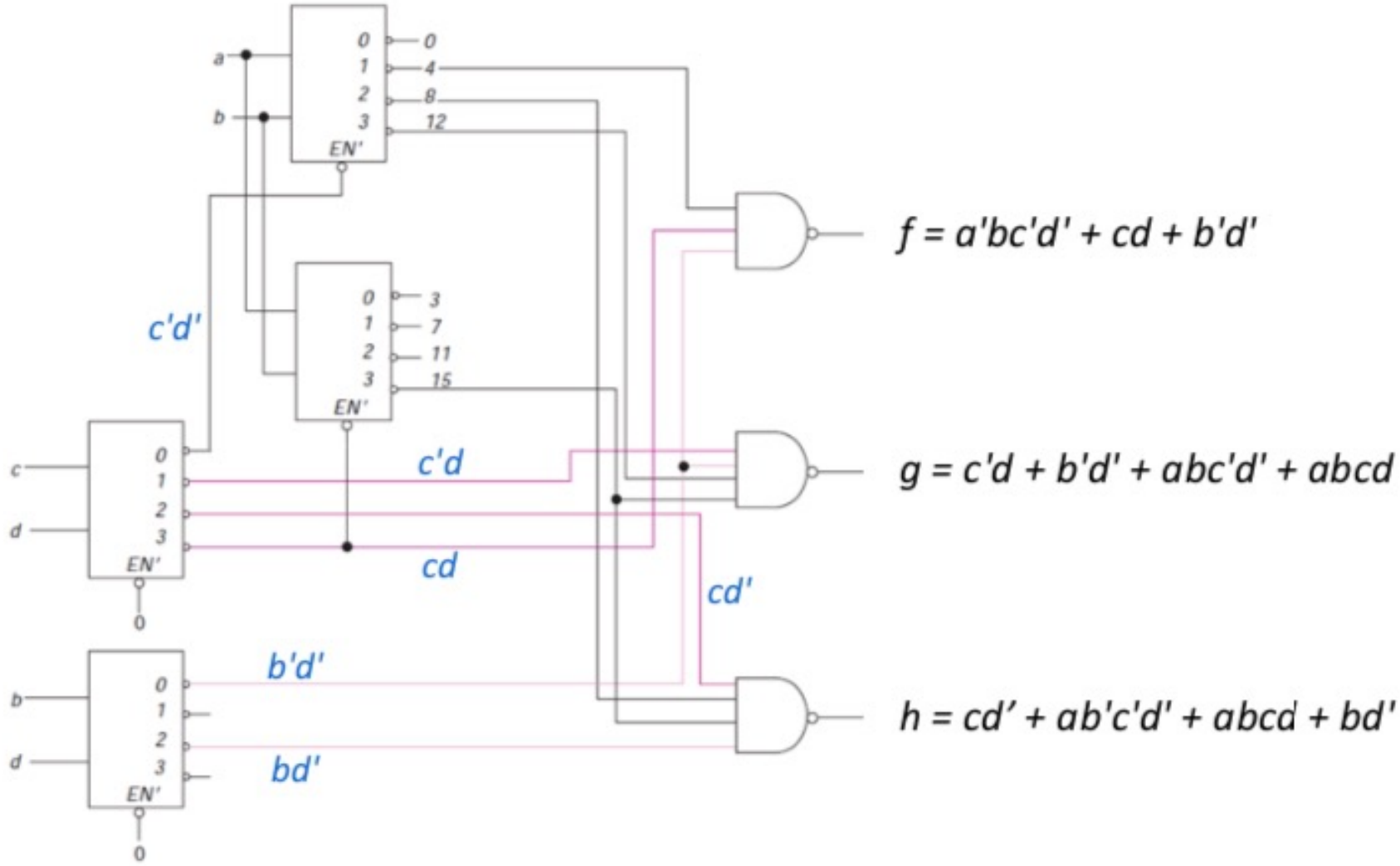
$h = cd' + ab'c'd' + abcd + bd'$

Example cont.

- Blue groups correspond to *cd*, *c'd*, and *cd'*, the 3, 1, and 2 outputs, respectively, from decoder with inputs *c* and *d*
- Similarly, red groups correspond to *b'd'* and *bd'*, the 0 and 2 outputs from decoder with inputs *b* and *d*
- Remaining 1's are covered as before, by adding a second-level decoder for the 0 and 3 outputs of the *cd* decoder
- Implementation of functions using only 4-input NAND gates, is shown next

f = a'bc'd' + cd + b'd' *g = c'd + b'd' + abc'd' + abcd* *h = cd' + ab'c'd' + abcd + bd'*

Example cont.



Encoders

- A binary encoder is the inverse of a binary decoder
- It is useful when one of several devices may be signaling a computer (by
- putting a 1 on a wire from that device)
  – Encoder then produces the device number
- An encoder has 2^n (or fewer) input lines and n output lines
- Output lines, generate the binary code corresponding to the input value
- If we can assume that exactly one input (of A_0, A_1, A_2, A_3) is 1, then TT of Table 5.4 describes behavior of the device

Encoders

- If, indeed, only one of the inputs can be 1, then this table is adequate

Table 5.4 A four-line encoder.

A_0	A_1	A_2	A_3	Z_0	Z_1
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

- Outputs can be expressed by:

$$Z_0 = A_2 + A_3$$
$$Z_1 = A_1 + A_3$$

Octal to Binary Encoder

- Octal to binary encoder can be obtained by first constructing TT

D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

- Outputs of the encoder can be expressed by the following functions:

$$X = D_4 + D_5 + D_6 + D_7$$
$$Y = D_2 + D_3 + D_6 + D_7$$
$$Z = D_1 + D_3 + D_5 + D_7$$

Problems with Encoders

- Problem 1: If two inputs are 1 simultaneously, output produces an undefined combination
  - e.g., if *D*3 and *D*6 are 1 simultaneously, output of encoder will be 111
    - Because all three outputs are equal to 1; but output 111 does not represent either binary 3 or binary 6
  - To resolve this ambiguity, encoder circuit must establish *input priority* to ensure that only one input is encoded
  - If we establish a *higher priority* for inputs with higher subscript numbers, and if both *D*3 and *D*6 are 1 at the same time, the output will be 110 (i.e. binary code of 6) because *D*6 has higher priority than *D*3

Problems with Encoders

- Problem 2: Another ambiguity is that an output with all 0's is generated when all the inputs are 0. But output 0 is also generated when *D*0 is equal to 1
  - This discrepancy has been resolved by providing one more output to indicate whether at least one input is equal to 1

D_0	D_1	D_2	D_3	D_4	D_5	D_6	D_7	X	Y	Z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Priority Encoder

- Both of previous problems can be resolved by the priority encoder
  - If more than one input can occur at the same time, then some priority must be established

- Priority encoder is an encoder circuit that includes priority function

Priority Encoder Operation

- Operation of priority encoder is such that
  – If two or more inputs are equal to 1 at the same time, the input having the highest priority (i.e. with highest subscript) will take the precedence

- Output would then indicate the number of the highest priority device with an active input

- Priorities are normally arranged in descending (or ascending) order with the highest priority given to the largest (smallest) input number

4 Input Priority Encoder

Truth table of a four-input priority encoder is as follows:

D_0	D_1	D_2	D_7	X	Y	V
0	0	0	0	0	0	0
1	0	0	0	0	0	1
X	1	0	0	1	0	1
X	X	1	0	1	1	1
X	X	X	1	1	1	1

Input D_3 has the highest precedence

- So, regardless of the values of other inputs, when this input is 1, the output for XY is 11 (binary of 3)

Priority Encoder

- Instead of listing all the 16 minterms of four variables, we have used an X in the input to represent either 0 or 1
  - For example, X100 represents the two minterms 0100 and 1100.
- In the output, variable *V* represents a valid bit indicator which is set to 1 when one or more inputs are equal to 1
- If all inputs are 0, there is no valid input and *V* is equal to 0, so other two outputs are not inspected and are specified as don't care conditions

Priority Encoder

- When we draw K-Maps for *X*, *Y*, and *V*, we get the following simplified functions

$$X = D2 + D3$$
$$Y = D3 + D1\ D'2$$
$$Z = D0 + D1 + D2 + D3$$

8 Input Priority Encoder

- TT is shown, where device 7 has the highest priority
- Output *NR* indicates that there are *No Requests*
  - In that case, we don't care what the other outputs are
- If device 7 has active signal (i.e., a 1), then the output is binary for 7, regardless of what other inputs are (as shown on second line of table)

Table 5.5 A priority encoder.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0

highest priority = A_7 (111)

8 Input Priority Encoder

$$Z_1 = A_7 + A_6A_7' + A_3A_4'A_5'A_6'A_7' + A_2A_3'A_4'A_5'A_6'A_7'$$

Using property P10a : $a+a'b = a+b$

$$Z_1 = A_7 + A_6 + A_3A_4'A_5'A_6'A_7' + A_2A_3'A_4'A_5'A_6'A_7'$$

$$= A_7 + A_6 + (A_3 + A_2A_3')A_4'A_5'A_6'A_7'$$

$$= A_7 + A_6 + (A_3 + A_2)A_4'A_5'A_6'A_7'$$

$$= A_7 + A_6 + (A_3 + A_2)A_4'A_5'A_7'$$

$$= A_6 + A_7 + (A_2 + A_3)\,A_4'\,A_5'$$

Table 5.5 A priority encoder.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0

8 Input Priority Encoder

Equations describing this deviceare

$$NR = A_0' A_1' A_2' A_3' A_4' A_5' A_6' A_7'$$
$$Z_0 = A_4 + A_5 + A_6 + A_7$$
$$Z_1 = A_6 + A_7 + (A_2 + A_3) A_4' A_5'$$
$$Z_2 = A_7 + A_5 A_6' + A_3 A_4' A_6' + A_1 A_2' A_4' A_6'$$

Table 5.5 A priority encoder.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0

Active Low BCD Encoder

- 74147 is a commercial BCD encoder
  - Taking 9 active low inputs and encoding them into 4 active low outputs
  - Input lines are numbered *9'* to *1'* and the outputs are *D'*, *C'*, *B'*, and *A'*
  - Note that all outputs of 1 (inactive) indicate that no inputs are active and there is no *0'* input line

Table 5.6 The 74147 priority encoder.

1'	2'	3'	4'	5'	6'	7'	8'	9'	D'	C'	B'	A'
1	1	1	1	1	1	1	1	1	1	1	1	1
X	X	X	X	X	X	X	X	0	0	1	1	0
X	X	X	X	X	X	X	0	1	0	1	1	1
X	X	X	X	X	X	0	1	1	1	0	0	0
X	X	X	X	X	0	1	1	1	1	0	0	1
X	X	X	X	0	1	1	1	1	1	0	1	0
X	X	X	0	1	1	1	1	1	1	0	1	1
X	X	0	1	1	1	1	1	1	1	1	0	0
X	0	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	0

highest priority = 9' (1001)' = 0110

To Be Continued